POPULAR MEDICAL COLLEGE & HOSPITAL LTD · HEALTHCARE ERP (HIS)

Test Case Document

Playwright automation suite — Login, Home launcher, Patient Registration, Patient Admission, the admission dashboard, the end-to-end patient journey, and the five Nurse Station tabs: Medicine Indent, Consultancy Service, IPD Service Entry, Diet Indent and Diet Dashboard. 138 cases, all automated.

1. Document control

Document	Test Case Document — PMCH Healthcare ERP Test Automation
Version	2.3
Date	22 October 2026
Prepared by	QA Automation — EMSL Dev Team
Application under test	Popular Medical College and Hospital Limited — HIS ERP (Blazor / MudBlazor)
Automation framework	Playwright Test 1.63 · TypeScript · Page Object Model
Source of truth	`tests/` in this repository. Every case below maps to one `test()` — each section names its spec file, and each row gives the line that case starts on. The **Actual result** and **Status** columns are read from the Playwright report for the run named below, not written by hand.
Results reflected	Full regression run of **05 Oct 2026, 15:28 UTC** — 99 tests in 33 m 33 s: **78 passed**, **16 known fail** (cases marked `test.fail()`, each watching a defect in the companion bug report), **2 failed** and **3 skipped**. The smoke and API suites were not part of that cycle and are marked NOT RUN. **§20, IPD Service Entry, is from a separate run of 06 Oct 2026** — 27 cases, **13 passed** and **14 known fail**, no unexpected failures and no skips. That suite is new; every other suite is unchanged from the 05 Oct cycle.
Companion document	`docs/BUG_REPORT.pdf` — the 37 defects raised against these cases, two of them now closed, with screenshots and video.

2. Scope and environment

In scope	Authentication, module launcher, Patient Registration list and create-patient wizard, New Admission wizard (Patient / Contact Person / Advance Payment / Admission Detail), the admission dashboard — looking an admission up and printing its paperwork — host availability, one end-to-end register-then-admit-then-indent flow, and the five **Nurse Station** tabs the ward works from: Medicine Indent and its Verify Indent pane, Consultancy Service (the cart, the bill and the cancel-with-reason dialog), IPD Service Entry (the cart, the bill and the entry row, up to but not including the Save — see §20), Diet Indent, and Diet Dashboard (the date range, the search, both grids, and Stop / Continue).
Out of scope	OPD, Pharmacy, Diagnostic, HR, Accounts and Billing modules; the ID-card print and the admission dashboard's list-level *Print List* / *Print Package List* buttons; what is *inside* a printed document, which is checked as a valid, non-empty PDF and no further; role-based permissions beyond the Administrator account; performance and security testing.
Environment	ERP web application over HTTPS on the hospital host (`BASE_URL` in `.env`; self-signed certificate, so `ignoreHTTPSErrors` is on). Live database — see the warning in section 5.
Browser	Chromium (Desktop Chrome profile) at 1920×1080, headed, one worker. Firefox and WebKit projects exist in `playwright.config.ts` but are commented out; mobile viewports are not enabled.
Test account	Administrator account read from `ERP_USERNAME` / `ERP_PASSWORD`. Credentials never appear in source control or in this document.
Session handling	`tests/auth.setup.ts` logs in once and saves the browser state to `playwright/.auth/user.json`. Signed-in suites reuse it; signed-out suites clear it with `test.use({ storageState: { cookies: [], origins: [] } })`.
Timeouts	90 s per test, 15 s per assertion, 30 s per action, 45 s per navigation — raised from the Playwright defaults because the host is slow. Three cases raise their own further (noted in their rows).

3. Legend

Expected vs actual	**Expected result** is what the case asserts — written against how the module is supposed to behave, never trimmed to match how it does. **Actual result** is what the 05 Oct 2026 run observed. AS EXPECTED means every assertion in the expected column held; anything else is spelled out, and names the defect it was raised as. The sixteen KNOWN FAIL cases are the ones where the two columns deliberately disagree.
Priority	**P1** — blocks the release if it fails (login, core patient flow). **P2** — important, has a workaround. **P3** — cosmetic or defensive.
Status	PASS passed in the 05 Oct 2026 run · KNOWN FAIL failed *as designed* — the case is marked `test.fail()` and asserts the behaviour the module is supposed to have, so the run reports it as expected-to-fail and will report it as *unexpectedly passing* the day the defect is fixed; the defect it watches is named in the cell · FAIL failed unexpectedly, defect raised · SKIPPED the live ward had no data the case needed · NOT RUN not part of that cycle
Writes to live data	Rows marked WRITES leave a real record in the hospital database. Every other case is read-only.

4. Coverage summary

SUITE	SPEC FILE	CASES	PASS	KNOWN FAIL	FAIL	SKIPPED	NOT RUN	SESSION
API — Health	`tests/api/health.spec.ts`	2	—	—	—	—	2	None
Setup — Authentication	`tests/auth.setup.ts`	1	1	—	—	—	—	Creates it
Smoke — Login	`tests/smoke/login.smoke.spec.ts`	5	—	—	—	—	5	Signed out
Smoke — Admission	`tests/smoke/admission.smoke.spec.ts`	4	—	—	—	—	4	Signed in
Regression — Login	`tests/regression/login.regression.spec.ts`	7	7	—	—	—	—	Signed out
Regression — Home	`tests/regression/home.regression.spec.ts`	7	7	—	—	—	—	Signed in (007 signed out)
Regression — Registration	`tests/regression/registration.regression.spec.ts`	6	6	—	—	—	—	Signed in
Regression — Admission	`tests/regression/admission.regression.spec.ts`	2	2	—	—	—	—	Signed in
Regression — Patient journey	`tests/regression/admission.regression.spec.ts`	1	1	—	—	—	—	Signed out
Regression — Admission dashboard	`tests/regression/admission-dashboard.regression.spec.ts`	7	7	—	—	—	—	Signed in
Regression — Nurse Station	`tests/regression/nurse-station.regression.spec.ts`	11	11	—	—	—	—	Signed in
Regression — Consultancy Service	`tests/regression/consultancy-service.regression.spec.ts`	23	14	9	—	—	—	Signed in
Regression — Diet Indent	`tests/regression/diet-indent.regression.spec.ts`	16	15	1	—	—	—	Signed in
Regression — Diet Dashboard	`tests/regression/diet-dashboard.regression.spec.ts`	19	10	6	—	3	—	Signed in
Regression — IPD Service Entry	`tests/regression/ipd-service-entry.regression.spec.ts`	27	14	13	—	—	—	Signed in
Total		**138**	**95**	**29**	**—**	**3**	**11**	

The 126 cases that ran are the setup plus every regression suite. **Known fail** is not a red build: all 30 are cases marked `test.fail()`, which Playwright counts as expected — 29 of them after the 22 Oct pass closed two. The two red cases of that cycle were DD-02 and DD-13, both BUG-018. **Both are green as of 10 Oct 2026** — see the next note. The three skips are not failures: DD-03, DD-04 and DD-08 skip when the ward has not ordered a meal yet, which is the right answer at that hour and not downstream of anything.

Fix pass of 10 Oct 2026 — three red cases, two causes, one of them ours.

- **DD-02 and DD-13** asserted that the application's idea of today equalled the *test host's* clock. BUG-018 turned out not to be a Diet Dashboard defect at all: the ERP host runs about twelve hours fast — `+718 minutes` on three consecutive readings, the signature of `UTC+6` configured as `UTC−6` — so from local noon onwards the app's "today" is tomorrow and both cases failed over something neither is about. They now read the day off the dashboard itself and assert what the tab owns; the clock assertion moved to `DD-18`, which measures the skew off the host's HTTP `Date` header and so is wrong at every hour rather than half of them. BUG-018 stays open, retitled, and is now held the way every other defect here is.
- **TC_ADMD_006** was a defect in the automation, not in the ERP. The admission grid filters over the Blazor circuit in a little over 200 ms and keeps the previous rows on screen throughout, and the case's own guard did not cover that gap: every case in that suite seeds itself from the *first row of the unfiltered grid*, so `rowFor(seed)` was already visible before the filter had done anything. The case then read ten unfiltered rows, found nine of them in other beds, and failed — and passed whenever the host answered inside the gap, which is why it looked intermittent. `searchFor()` now waits for the grid to settle on *which* admissions it lists, not on how many: at ten rows a page, a count cannot tell one set of ten from another.

Verification pass of 22 Oct 2026 — two defects closed by the ERP, two found in the suite.

- **BUG-018 is fixed.** The host clock read `+0` against this machine, down from `+718` minutes. `DD-18` reported as *unexpectedly passing* — exactly what its marker was for — and the marker came off. The assertion stays as a live guard: a clock is the kind of thing that is corrected once and comes back after a reboot. Note that records written while it was wrong still carry the shifted stamp; correcting the clock does not correct those.
- **BUG-037 is fixed, and more besides.** The bill grid's `End End` now reads `End Date`, and the rest of the header row went with it: `S.Date` → `Service Date`, `Served_By` → `Served By`, `Service Change` → `Service Charge`, `Inv.No` → `Invoice No`, `CabinNo` → `Cabin No`, `OTName` → `OT Name`. Two messages were reworded with them: `Please input items` → `Please add at least one service.` and `Please input correct Qty.` → `Please enter a valid Qty.` Behaviour did not change anywhere; IS-01, IS-06 and IS-25 caught the wording, which is what pinning it is for. `Servedby` → `Served By` on Consultancy Service was part of the same pass and would have taken CS-01 … CS-23 down on the next full run; both page objects were updated.
- **IS-26 could never have passed**, and only looked sound while the typo was there to fail on. It read the headers with `allInnerTexts()`, which applies the CSS that uppercases them, so it was comparing `END DATE` against `End Date` and would have gone on failing after the fix. It now reads `allTextContents()`, the markup's own casing — which is what IS-01 was doing all along, and why IS-01 passed on the day IS-26 did not.
- **IS-09 was asserting the wrong thing.** It read a catalogue rate once, then re-searched the lookup before each of its two lines and picked the row at the same index — but the lookup does not guarantee the order, so the second line went in at a different service's rate and the case failed on a total of 5000 against an expected 3000. It now checks each line against what the cart says it cost. Its subject was never the catalogue's arithmetic (IS-05 covers that) but that the bin takes off exactly one line and Total Amount follows.

IPD Service Entry (§20) was run on 06 Oct 2026, after the figures above were taken; the other suites are from the 05 Oct cycle and are unchanged. Its 27 cases ran in two passes — the first was cut short by the test host running out of memory, not by the application, and the remaining cases were run again to completion. Two defects in the automation itself were found and fixed in that first pass and are worth recording, because both were the suite's fault and neither is an application defect: the hidden Service Date input posts `2026-10-06 16:23` for a value the app set and `2026-10-03T10:00:00` for one a nurse typed, so IS-17 was comparing formats rather than moments; and a Service lookup search can be swallowed before Blazor wires the button, leaving the unfiltered catalogue on screen, which failed IS-04 once. The page object now reduces both date shapes to a moment before comparing, and re-sends a search whose grid did not move.

5. Data safety

The application under test is the hospital's **production** ERP. Every case marked `WRITES` leaves a real record against a real patient. They fall into three groups.

CASES	WHAT THEY LEAVE BEHIND	HOW IT IS CLEARED
`TC_FLOW_001` `TC_SMK_ADM_004`	A registered patient, an admission holding a real bed and real advance payments, and the first medicine indent against it. TC_SMK_ADM_004 fills the wizard but stops short of the submit.	**Nothing in the suite undoes them.** Cancelling an admission is its own workflow under **Hospital > Admission Cancel**. Run TC_FLOW_001 deliberately, and expect a `Masud Rana Test-<n>` patient after each run.

`TC_NS_004, TC_NS_005`
`DI-04, DI-05, DI-09,`
`DI-12 … DI-15`
`DD-14 … DD-17`
	Medicine and diet indents filed against live admissions. They land as **Pending**, where the pharmacy and the canteen see them.	Not undone by the suite — an indent is cancelled from **Nurse Station > Verify Indent**. Every diet indent carries a mark unique to its case in Remarks, so they can be told from a nurse's. DD-15 … DD-17 move diets between the two dashboard grids and *do* put the ward back as they found it, acting only on indents they filed themselves.

`CS-19 … CS-22`	Consultancy services charged to a live admission's bill.	**Each takes its own charges back off** through the tab's cancel dialog, in a `finally`, so a failure mid-case still cleans up — a cancellation with a stated reason, recorded in IPD Bill Change History, not a deletion. Only the rows the case added are taken off; on a long-stay patient the rest of the bill is a month of real treatment. Three of the four prefer a service the catalogue prices at zero.

Every other case is read-only: the registration and admission cases deliberately stop before any Confirm button, the negative cases on the indent forms are negative precisely because the app refuses them, and the dashboard cases only search, sort and print — a printed document changes nothing in the record it is printed from.

IPD Service Entry (§20) writes nothing at all, and that is a finding rather than caution. Its bill grid carries no bin, no edit and no cancel (IS-24 / BUG-027), so a service saved there cannot be taken off again from the tab — there is no equivalent of the cancel dialog that makes CS-19 … CS-22 safe. All 27 of its cases therefore stop at the cart, which is client-side state and reaches no bill. The single exception is IS-25, which presses Save against a cart it has first *asserted* to be empty, so the only outcome available to it is the refusal it is testing for. The tab's Save, its OTName picker and its Print All button are consequently uncovered, and listed under Known coverage gaps (§21).

Both writing suites are in a `describe` of their own, so a run that should not write can leave them out:

```
npx playwright test tests/regression/consultancy-service.regression.spec.ts -g "CS-0|CS-1|CS-23"
npx playwright test tests/regression/diet-indent.regression.spec.ts -g "DI-0[12368]|DI-1[01]"
npx playwright test tests/regression/diet-dashboard.regression.spec.ts -g "DD-0|DD-1[0-3]"
```

6. API — Health

Spec: `tests/api/health.spec.ts` · Project `api` · no browser, no session.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_API_001 line 6	Health endpoint reports the host is healthy	ERP host reachable at `API_URL`.	1. Send `GET /health` with the API request context. 2. Read the status code and the response body.	`GET /health`	• Status is **200**. • Body, trimmed, is exactly `Healthy`.	Not run in this cycle.	P1	NOT RUN
TC_API_002 line 13	Login page is served by the identity provider	ERP host reachable at `API_URL`.	1. Send `GET /Account/Login`. 2. Read the status code and the HTML body.	`GET /Account/Login`	• Status is **200**. • Body contains `Input.Email`, proving the sign-in form rendered.	Not run in this cycle.	P1	NOT RUN

7. Setup — Authentication

Spec: `tests/auth.setup.ts` · runs before every UI project; if it fails, every signed-in case is skipped.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_SETUP_001 line 10	Log in once and persist the session for the suite	`.env` holds valid `ERP_USERNAME` / `ERP_PASSWORD`.	1. Open `/Account/Login`. 2. Enter the administrator username and password; submit. 3. Wait for the module launcher to load. 4. Save the browser storage state to disk.	Administrator account from `.env`	• Launcher loads with the tenant name and user role. • `playwright/.auth/user.json` is written and reused by every signed-in project.	AS EXPECTED The session was saved to `playwright/.auth/user.json` and reused by every signed-in project in the run.	P1	PASS 3.1 s

8. Smoke — Login

Spec: `tests/smoke/login.smoke.spec.ts` · signed out — the suite clears the saved session so nothing is inherited.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_SMK_LOGIN_001 line 12	Login page renders every field	No session.	1. Open the login page. 2. Wait for the form to load. 3. Inspect each control.	—	• Username placeholder is *Enter your username*. • Password placeholder is *Enter your password*. • Submit button reads *Log in*. • Remember-me and Forgot-password are visible.	Not run in this cycle.	P1	NOT RUN
TC_SMK_LOGIN_002 line 23	Valid credentials land on the module launcher	No session; administrator account is active.	1. Open the login page. 2. Enter the administrator username and password; submit. 3. Wait for the launcher.	Administrator account from `.env`	• Launcher is displayed. • Tenant reads *Popular Medical College and Hospital*. • Role reads *Administrator*.	Not run in this cycle.	P1	NOT RUN
TC_SMK_LOGIN_003 line 35	Invalid password is rejected	No session.	1. Open the login page. 2. Enter a known username with a deliberately wrong password; submit.	`admin` / `wrong-password-on-purpose`	• Invalid-login error is shown. • User stays on the login page; no session is created.	Not run in this cycle.	P1	NOT RUN
TC_SMK_LOGIN_004 line 42	Unknown user is rejected	No session.	1. Open the login page. 2. Enter a username that does not exist with any password; submit.	`no-such-user-12345` / `whatever`	• Invalid-login error is shown. • The message does not reveal whether the account exists.	Not run in this cycle.	P1	NOT RUN
TC_SMK_LOGIN_005 line 49	Unauthenticated visit redirects to login	No session.	1. Navigate straight to the application root `/`.	—	• URL becomes `/Account/Login`. • The login form is rendered.	Not run in this cycle.	P1	NOT RUN

9. Smoke — New Admission wizard

Spec: `tests/smoke/admission.smoke.spec.ts` · signed in on the saved session · page `/hospital/patientadmission`. Nothing here presses *Create Patient Admission*; the end-to-end submit is TC_FLOW_001.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_SMK_ADM_001 line 18	Wizard opens on the Patient tab with its required fields ready	Signed in as Administrator.	1. Open the New Admission wizard. 2. Wait for the page to load. 3. Inspect the Patient tab and its fields.	—	• Patient tab is active. • UHID is editable. • Full Name, Surname, Patient Mobile are editable; DOB and Blood Group are visible.	Not run in this cycle.	P1	NOT RUN
TC_SMK_ADM_002 line 34	The four marked tabs open in order	Signed in; wizard loaded.	1. Open **Patient** — check Full Name rendered. 2. Open **Contact Person** — check Guardian Name and Relation. 3. Open **Advance Payment** — check Select Card, BKash Payment, Payment (Cash). 4. Open **Admission Detail** — check Select Department, Admitted to, Bed/Cabin.	—	• Each tab goes active when opened. • The pane behind each tab renders its own fields — a tab that switches without its fields arriving still fails.	Not run in this cycle.	P1	NOT RUN
TC_SMK_ADM_003 line 57	A patient loads by UHID into the Patient tab	Signed in; at least one patient in the Patient Registration grid for the selected date range.	1. Open Patient Registration and read the UHID from the first grid row. 2. Open the New Admission wizard. 3. Search that UHID.	UHID read live from the grid (10+ digits)	• Full Name, Patient Mobile and DOB are all filled from the registration record — none is empty.	Not run in this cycle.	P1	NOT RUN

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
`TC_SMK_ADM_004` `line 80`	The tabs take entry in order, up to the submit **WRITES**	Signed in; wizard loaded. Test timeout raised to 240 s.	1. Fill the Contact Person tab; confirm Guardian Name holds what was typed. 2. Fill the Advance Payment tab (card, bKash, cash). 3. Read the page's own *Total:* panel. 4. Press Next into Admission Detail; choose Department and Admitted-to. 5. Wait for the bed list to populate, pick the first real bed. 6. Search and pick an assigned doctor.	Guardian `Test-<n>` · Card VISA 1000, bKash 2000, Cash 3000 · HDU / HDU · doctor search `Dr`	• Total panel shows **6000** — the app's own sum, which is the honest proof all three amounts registered. • Bed list loads more than the placeholder option and a bed is selected. • Assigned doctor contains the searched text. • *Create Patient Admission* is enabled — and deliberately not pressed.	Not run in this cycle.	P1	NOT RUN

10. Regression — Login

Spec: `tests/regression/login.regression.spec.ts` · signed out. The form relies on HTML5 `required` rather than a server-rendered validation summary, so the empty-field cases assert constraint validity plus "the form was never submitted" instead of hunting for an error banner.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_LOGIN_001 line 16	Login page is displayed with all required elements	No session.	1. Open the login page and wait for it to load. 2. Inspect each field, the submit button and the error area.	—	• Username editable, placeholder *Enter your username*. • Password editable, `type="password"`, placeholder *Enter your password*. • Submit enabled and reads *Log in*. • Remember-me and Forgot-password visible; no error banner.	AS EXPECTED	P1	PASS 3.0 s
TC_LOGIN_002 line 79	Valid credentials land on the home page	No session; administrator account active.	1. Open the login page. 2. Enter the administrator username and password; submit. 3. Wait for the launcher.	Administrator account from `.env`	• Launcher loads. • Tenant reads *Popular Medical College and Hospital*; role reads *Administrator*. • No error banner.	AS EXPECTED	P1	PASS 2.9 s
TC_LOGIN_003 line 32	Invalid username with a valid password is rejected	No session.	1. Open the login page. 2. Enter a non-existent username with the real password; submit.	`no_such_user_9912` + real password	• Invalid-login error is shown; the user is not signed in.	AS EXPECTED	P1	PASS 3.2 s
TC_LOGIN_004 line 41	Valid username with an invalid password is rejected	No session.	1. Open the login page. 2. Enter the real username with a wrong password; submit.	Real username + `WrongPass!123`	• Invalid-login error is shown; the user is not signed in.	AS EXPECTED	P1	PASS 3.9 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_LOGIN_005 line 50	Empty username is blocked by field validation	No session.	1. Open the login page. 2. Leave Username blank, fill Password; submit.	Username *(blank)* + real password	• Username reports a required-field constraint violation. • The form is never submitted — the browser stays on the login page.	AS EXPECTED	P2	PASS 2.8 s
TC_LOGIN_006 line 60	Empty password is blocked by field validation	No session.	1. Open the login page. 2. Fill Username, leave Password blank; submit.	Real username + password *(blank)*	• Password reports a required-field constraint violation. • The form is never submitted.	AS EXPECTED	P2	PASS 3.2 s
TC_LOGIN_007 line 70	Both fields empty are blocked by field validation	No session.	1. Open the login page. 2. Submit with both fields blank.	—	• Both fields report a required-field constraint violation. • The form is never submitted.	AS EXPECTED	P2	PASS 3.2 s

11. Regression — Home / module launcher

Spec: `tests/regression/home.regression.spec.ts` · signed in, except TC_HOME_007. The launcher is a MudBlazor shell, so module tiles are plain text nodes rather than links — they are matched exactly, because a substring match also hits the hidden language dropdown.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
`TC_HOME_001` line 19	Home page is displayed with all required elements	Signed in as Administrator.	1. Open the home page and wait for it to load. 2. Inspect the header chrome.	—	• Tenant name visible and not empty. • User name, user role and language selector all visible.	AS EXPECTED	P1	PASS 2.5 s
`TC_HOME_002` line 30	Core ERP modules are listed on the launcher	Signed in as Administrator.	1. Open the home page. 2. Look for each core module tile by exact name.	REGISTRATION, OPD, Pharmacy, Diagnostic, HR, ACCOUNT	• All six tiles are visible; the failure message names the missing module.	AS EXPECTED	P1	PASS 2.9 s
`TC_HOME_003` line 39	Tenant is the hospital the session belongs to	Signed in as Administrator.	1. Open the home page. 2. Read the tenant name in the header.	—	• Tenant matches *Popular Medical College and Hospital* (case-insensitive).	AS EXPECTED	P1	PASS 3.2 s
`TC_HOME_004` line 46	Signed-in user context is shown	Signed in as Administrator.	1. Open the home page. 2. Read the user role and user name.	—	• Role matches *Administrator*. • User name is not empty.	AS EXPECTED	P2	PASS 2.4 s
`TC_HOME_005` line 54	A module tile that does not exist is not rendered	Signed in as Administrator.	1. Open the home page. 2. Look for a module name that cannot exist.	`NO_SUCH_MODULE_9912`	• No such tile is rendered — negative control proving the tile matcher is not matching everything.	AS EXPECTED	P3	PASS 2.2 s
`TC_HOME_006` line 61	Reloading keeps the user on the launcher	Signed in as Administrator.	1. Open the home page. 2. Reload (tolerant of the host's HTTP 429 cool-off — see BUG-003). 3. Wait for the launcher again.	—	• Launcher loads again; the session is not lost. • The REGISTRATION tile is still visible.	AS EXPECTED	P2	PASS 2.9 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_HOME_007 line 78	An unauthenticated visitor is sent to the login page	Session deliberately cleared.	1. Navigate to the home page with no session.	—	• URL becomes `/Account/Login`. • The login form renders and the tenant header is hidden — no launcher content leaks.	AS EXPECTED	P1	PASS 2.7 s

12. Regression — Patient Registration

Spec: `tests/regression/registration.regression.spec.ts` · signed in. Follows the module the way a user reaches it — REGISTRATION tile on the launcher, which loads that module's menu into the left drawer, then PATIENT REGISTRATION in the drawer. Nothing here submits the create-patient wizard, because confirming a registration would leave a real patient behind on every run.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_REG_001 `line 23`	REGISTRATION tile loads the module menu into the sidebar	Signed in; on the launcher.	1. Open the home page. 2. Confirm the drawer carries no PATIENT REGISTRATION link yet. 3. Click the REGISTRATION tile.	—	• Before the click: the link genuinely does not exist (count 0). • After: the REGISTRATION group and the PATIENT REGISTRATION link appear, the link pointing at `/hospital/newregistration`.	**AS EXPECTED**	P1	**PASS** 4.5 s
TC_REG_002 `line 44`	PATIENT REGISTRATION opens the patient list	Signed in; on the launcher.	1. Click the REGISTRATION tile. 2. Click PATIENT REGISTRATION in the drawer.	—	• The patient list page loads. • URL ends with `/hospital/newregistration`.	**AS EXPECTED**	P1	**PASS** 4.3 s
TC_REG_003 `line 57`	Patient list is displayed with all required elements	Signed in; patient list open.	1. Open the patient list. 2. Inspect the filter bar and toolbar.	—	• Show From and Show To date filters visible; Show button enabled. • Add New visible and pointing at `/patients/new`. • Search box editable.	**AS EXPECTED**	P1	**PASS** 2.7 s
TC_REG_004 `line 71`	The grid lists patients under the expected columns	Signed in. **At least one patient registered within the grid's default date range** (today) — see BUG-001.	1. Open the patient list. 2. Read the header row. 3. Wait for the first data row.	Live hospital data	• Exactly six headers, in order: NAME, UHID, REG DATE, PHONE NUMBER, DOB, ACTION. • At least one data row is visible.	**AS EXPECTED**, but slow: **10.3 s** to render the first row, against 2.7 s for TC_REG_003 on the same page in the same session. It was 33.8 s on 03 Oct. Raised as **BUG-004**.	P1	**PASS** 10.3 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_REG_008 line 89	Add New opens the create-patient form	Signed in; patient list open.	1. Open the patient list. 2. Click Add New.	—	• The create-patient wizard loads. • URL ends with `/hospital/patients/new`.	AS EXPECTED	P1	PASS 4.3 s
TC_REG_009 line 102	The create-patient form opens on the Patient tab	Signed in; create-patient wizard open.	1. Open the create-patient wizard directly. 2. Read the `aria-selected` state of each tab.	—	• Patient tab is selected. • Contact Person and Corporate Client are not selected.	AS EXPECTED	P2	PASS 3.3 s

ID gap. TC_REG_005, TC_REG_006 and TC_REG_007 — the grid search and row-action cases — were removed from the spec and are not currently automated. They are listed as a coverage gap in section 15; the surviving IDs were left unrenumbered so that earlier runs and defect reports still line up.

13. Regression — New Admission

Spec: `tests/regression/admission.regression.spec.ts` · signed in. The wizard admits a patient who is already registered, so the UHID is read out of the registration grid rather than hard-coded — the grid holds live hospital data and no patient in it is guaranteed to still be there next week.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_ADM_001 `line 48`	New Admission opens from the Hospital module menu	Signed in; on the launcher.	1. Open the home page. 2. Open the Hospital module and find the New Admission link. 3. Click it.	Route `/hospital/patientadmission`	• The admission wizard loads. • UHID is editable and the Patient tab is active.	**AS EXPECTED**	P1	**PASS** 5.0 s
TC_ADM_002 `line 63`	Searching a UHID loads that patient into the wizard	Signed in. At least one patient registered in the last year — the list opens filtered to today (BUG-001), so the case widens the window itself.	1. Open Patient Registration and widen *Show From* to the last year. 2. Read the name and the UHID off the first grid row, in one go, so a re-render cannot hand back two different patients. 3. Open the New Admission wizard. 4. Search that UHID.	UHID and name read live from the grid	• Full Name in the wizard equals the name in the grid, normalised — the grid cell wraps the name and the wizard's field does not. • Patient Mobile and DOB are filled from the registration record.	**AS EXPECTED** This was the 03 Oct failure. It passes now that the case widens the date window itself — the automation defect behind that (AUT-001) is fixed. The list still opens filtered to today, so **BUG-001** stands.	P1	**PASS** 13.2 s — was the 03 Oct failure; **AUT-001** is fixed, **BUG-001** stands

14. Regression — Patient journey (end to end)

Spec: `tests/regression/admission.regression.spec.ts` · signed out, because logging in is the first thing it checks. One patient, followed the way the hospital actually takes one on. It is one test and not three because the admission wizard needs a patient who is registered and not already on a ward, and the only way to be sure of that is to have just registered them. Retries are disabled: a retry would register a second patient rather than tell us anything new.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
`TC_FLOW_001` `line 112`	Log in, register a patient, then admit them **WRITES**	No session. A free bed in the target ward. Test timeout raised to 600 s; retries disabled.	1. **Log in** with the administrator account and reach the launcher. 2. **Register the patient:** open the create-patient wizard, fill the Patient tab, confirm the DOB picker wrote `4/10/1998`, press Confirm Registeration and capture the new Patient ID. 3. **Admit them:** open the New Admission wizard and search the UHID just issued. 4. Fill Contact Person, then Advance Payment; read the page's own *Total:* panel. 5. Press Next into Admission Detail; fill department, ward, bed and assigned doctor. 6. Press Create Patient Admission and capture the confirmation text. 7. **Verify on the ward:** open Hospital > Dashboard (the admission dashboard) and search the patient name as registered; read the admission number off the row. 8. **Find it by number and print:** search that admission number, then press every print button in the row's Action cell, one after another, and read back the PDF each one produced.	Patient `Masud Rana Test-<n>`, Male, DOB 10 Apr 1998, A+, mobile `01876765454`, NID `98876756757`, Tangail / Bhuapur · Card VISA 1000, bKash 2000, Cash 3000 · HDU / HDU	• Login succeeds and the launcher loads. • Registration returns a Patient ID of 10 or more digits. • The wizard pulls up that exact patient — name and mobile match what was registered. • Total panel shows **6000**. • The confirmation carries no *error / failed / invalid* wording. • An admission row exists on the admission dashboard: admission number of 10+ digits, carrying the patient's mobile and a bed in the ward admitted to. • Searching that admission number returns exactly one row, and it is this patient. • At least four print buttons, each producing a PDF (`%PDF-` header, over 1 KB).	**AS EXPECTED** throughout: the patient was registered, admitted, indented for, and found on the dashboard; the Total panel read **6000**; every print button produced a PDF. The advance payments did not reach the Advance Dashboard — which this case does not assert, and which is **BUG-002**.	P1	**PASS** 1 m 48 s

Deliberately not asserted: that the advance payments reach the admission. On the runs so far they did not — Hospital > Advance Dashboard carried no row for the admission this flow created, even though the wizard summed the three amounts to 6000 before the submit. The case asserts only what it can stand behind, and the gap is raised separately as **BUG-002**.

15. Regression — Admission dashboard

Spec: `tests/regression/admission-dashboard.regression.spec.ts` · signed in. Route `/hospital/patientlist-dashboard` — Hospital > **Dashboard**, the list of everyone currently admitted. This is where an admission is looked up after the fact, so the cases are the two things the page is used for: finding one admission in it, and printing that admission's paperwork. The grid holds live hospital data, so no case hard-codes an admission — each reads one out of the first row and searches for what it just read. Read-only throughout.

One trap, recorded here because it is not a defect: the NAME column prints the patient's title in front of the name (`Mr Jalal Miah`) but the search box matches the name as it was registered, without it. Searching a name copied straight out of the grid returns nothing; `Jalal Miah` returns the row. TC_ADMD_004 strips the title before searching.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_ADMD_001 line 40	Dashboard opens from the Hospital module menu	Signed in; on the launcher.	1. Open the home page. 2. Open the Hospital module and find the Dashboard link in the drawer. 3. Click it.	Route `/hospital/patientlist-dashboard` — the link is reached by href, because it is labelled just "Dashboard"	• The admission dashboard loads with rows in the grid. • Search is editable; *Print List* and *Print Package List* are on screen.	AS EXPECTED	P1	PASS 5.4 s
TC_ADMD_002 line 59	The grid lists admissions under the expected columns	Signed in. At least one patient admitted.	1. Open the admission dashboard. 2. Read the grid's header row. 3. Read the ADM.NO and CABINNO cell of every row on the page.	Eleven columns: ADM.NO, ADM.DATE, ADM.TIME, NAME, CABINNO, ASSIGNED DOCTOR, PHONE, ADDRESS, CABINTYPE, PACKAGE, ACTION	• Exactly those eleven headers, in that order. • No row is missing its admission number or its bed — the two every admitted patient has.	AS EXPECTED	P2	PASS 3.5 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_ADMD_003 line 79	Searching an admission number finds that admission	Signed in. At least one patient admitted.	1. Open the admission dashboard and read the first row. 2. Type its admission number into Search.	Admission number read live from the grid	• Exactly one row comes back — an admission number is unique. • Its admission number and patient name are the ones read in step 1.	AS EXPECTED	P1	PASS 4.1 s
TC_ADMD_004 line 93	Searching a patient name finds their admission	Signed in. At least one patient admitted.	1. Open the admission dashboard and read the first row. 2. Drop the title from the name (`Mr Jalal Miah` → `Jalal Miah`). 3. Type the remainder into Search.	Patient name read live from the grid	• The admission read in step 1 is among the rows returned. • Its name matches. *Not* asserted as a single row: a ward holding two patients of one name is ordinary.	AS EXPECTED	P1	PASS 4.1 s
TC_ADMD_005 line 111	Searching a phone number finds that admission	Signed in. Skipped if the admission at the top of the grid carries no phone number.	1. Open the admission dashboard and read the first row. 2. Type its phone number into Search.	Phone number read live from the grid	• The admission read in step 1 is among the rows returned. • Its PHONE cell is the number searched. Not a single-row assertion — a family shares a number.	AS EXPECTED	P2	PASS 3.7 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_ADMD_006 line 125	Searching a cabin number finds the patient in that bed	Signed in. At least one patient admitted.	1. Open the admission dashboard and read the first row. 2. Type its cabin number into Search.	Cabin number read live from the grid, e.g. `HCU-27`	• The admission read in step 1 is among the rows returned. • Every row returned is for that bed — a bed holds one patient at a time.	**AS EXPECTED** Was intermittently red until 10 Oct 2026, and the fault was the suite's. The grid takes a little over 200 ms to filter and shows the old rows meanwhile, and this is the only case in the suite that asserts something about *every* row — the others read one row, the seed's own, which is visible either way. It was reading the unfiltered page. `searchFor()` now settles the grid first. The application was never at fault: a ward-wide check of 250 admissions found no bed name appearing anywhere outside its own column.	P2	**PASS** 6.5 s
TC_ADMD_007 line 143	Every Action print button renders a PDF	Signed in. At least one patient admitted. Test timeout raised to 240 s — four documents, the largest over a megabyte.	1. Open the admission dashboard and search the first row's admission number. 2. Press the first print button in the Action cell; wait for the tab it opens. 3. Read the document back out of the browser and close the tab. 4. Repeat for every remaining print button, one at a time, left to right.	One admission, read live from the grid. Measured on 03 Oct 2026: 1 388 783 / 643 337 / 37 660 / 643 270 bytes	• At least four print buttons, each opening its own tab. • Each tab carries a real PDF — `%PDF-` header, over 1 KB. • Not all four are the same file: these are different documents, and four identical ones would mean the buttons share a handler.	**AS EXPECTED**	P1	**PASS** 10.2 s

How a printed document is checked. The app does not navigate to a document or download one — it builds the file in the browser and opens a tab on a `blob:` URL, which renders in the browser's own PDF viewer and has no readable page behind it. The blob belongs to the dashboard, which created it, so the test fetches that URL from the dashboard and gets the actual bytes. That is what makes "the button printed something" checkable rather than just "a tab opened" — a page that failed to render would open a tab too.

Not clicked: some rows carry a cloud-arrow-down button among the print buttons. It produced nothing observable on any row tried — no tab, no download, nothing on screen — so the print run matches on the printer icon and steps over it rather than hanging on a document that never comes. Worth a question to the module owners; it is not raised as a defect because

what it is meant to do is unknown.

16. Regression — Nurse Station (Medicine Indent)

Spec: `tests/regression/nurse-station.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, which opens on **Medicine Indent > Add Indent** — where the ward nurse raises a medicine indent against an admitted patient: find the patient by admission number, set a priority, list the drugs and quantities, save. The cases are that flow end to end (TC_NS_004, confirmed on Verify Indent by TC_NS_005), the five ways the form is supposed to refuse, and the Verify Indent tab the pharmacy works from (TC_NS_011).

No case hard-codes a patient. Which beds are occupied changes by the hour, so each case reads a patient out of the live ward list and works with whoever that turns out to be. **TC_NS_004 and TC_NS_005 write** — each files a real indent that reaches the pharmacy as Pending; see section 5.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_NS_001 line 42	Nurse Station opens from the Hospital module menu	Signed in; on the launcher.	1. Open the home page and the Hospital module drawer. 2. Find the Nurse Station link by href and click it.	Route `/hospital/nurse-station`	• The page loads with **MEDICINE INDENT** and **ADD INDENT** both active — the indent form is on screen with nothing clicked. • Save is visible.	AS EXPECTED	P1	PASS 4.9 s
TC_NS_002 line 61	The Add Indent form offers every control the indent needs	Signed in; Nurse Station open.	1. Assert every tab in the strip is present. 2. Open the Priority dropdown and count each priority the ward can raise under. 3. Assert Outlet, OT Room, the Product Code lookup and the Quantity box. 4. Read the item grid's headers one by one.	Six priorities from `NurseStationPage.PRIORITIES`; item columns from `ITEM_COLUMNS`	• Every tab, every priority, and every named item column is present, in order. • Quantity is editable. A missing priority would silently narrow what the nurse can file.	AS EXPECTED	P2	PASS 3.4 s
TC_NS_003 line 98	Searching an admission number opens that patient on the indent form	Signed in; at least one patient on the ward.	1. Read the admission number off the first ward card. 2. Type it into the ward search. 3. Click the card that comes back and read the patient panel.	Admission number read live off the ward list	• The search returns **exactly one** card — an admission number belongs to one admission. • *Indent for* carries the patient's own UHID, not the admission number searched for. • Name, Cabin and Admitted DateTime are all filled from the admission record.	AS EXPECTED	P1	PASS 4.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_NS_004 line 118	A ten-line medicine indent saves against an admitted patient WRITES	Signed in; a patient on the ward. Test timeout raised to 420 s — ten lookups, ten picks, ten server round trips.	1. Select the first ward patient; choose a priority. 2. Add ten medicines through the Product Code lookup, each at its own quantity. 3. Read the item table back and compare it against what was ordered. 4. Press Save and read the confirmation.	Ten lines from `medicineIndent(10)` — one search term, a different drug per line, random quantities per run	• Priority takes: the dropdown turns `is-valid`. • Ten **distinct** medicines are listed, each in the quantity typed, in the order added. • Save answers *successful save*. • The form empties and Priority falls back to `is-invalid` — a table still holding ten lines would mean the confirmation was shown over an indent that never left.	AS EXPECTED Ten distinct medicines, each at the quantity typed, in order; saved, and the form cleared itself. The indent filed is recorded as a run annotation — patient, bed, priority and every line.	P1	PASS 1 m 38 s
TC_NS_005 line 165	A saved indent appears on Verify Indent as Pending WRITES	Signed in; a patient on the ward. Test timeout raised to 240 s.	1. Read the last indent number issued before this case files its own. 2. File a two-line indent against a ward patient. 3. Open Verify Indent and wait for an indent numbered above the baseline.	Two lines from `medicineIndent(2)`. Indent numbers run in sequence, which is the only handle there is — the save confirms itself with a snackbar and no number	• The indent reaches the pharmacy under the bed it was raised for, at the priority it was raised at. • Status is **Pending** — raised, not yet dispensed. • It carries a user name, and a number above the baseline.	AS EXPECTED The indent reached Verify Indent as **Pending**, under the bed it was raised for, numbered above the baseline.	P1	PASS 36.5 s
TC_NS_006 line 198	An admission number nobody is admitted under returns no patient	Signed in; Nurse Station open.	1. Search an admission number the ward could have had and does not. 2. Assert the ward list and the patient panel. 3. Clear the search and assert the list comes back.	`99999999999` — shaped like a real admission number, so the search is tested rather than the input validation	• No cards come back, and the patient panel stays empty rather than keeping whoever was last on it. • Clearing the box restores the list — a filter, not a failed fetch.	AS EXPECTED	P2	PASS 5.0 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_NS_007 line 218	Save with no patient and no priority is refused	Signed in; the form untouched.	1. Press Save on a form with nothing on it.	—	• The form answers in the markup rather than in a message: Priority is left marked `is-invalid`. • No patient, no item rows. An indent saved with no patient would be an indent nobody can serve.	AS EXPECTED	P1	PASS 4.1 s
TC_NS_008 line 234	Save with a patient but no medicines is refused	Signed in; a patient on the ward.	1. Select a ward patient and choose Routine. 2. Press Save with an empty item table.	Priority `Routine`	• Refused with *please fill medicine table*. • Refused, **not reset** — the patient and the priority survive, so the nurse adds the drugs and saves rather than starting over.	AS EXPECTED	P1	PASS 8.6 s
TC_NS_009 line 250	A medicine with a quantity of zero is not added to the indent	Signed in; a patient on the ward.	1. Select a patient, choose Routine, pick a medicine. 2. Commit the line at a quantity of **0**. 3. Commit the same medicine again at a real quantity.	Quantity 0, then a real quantity from `medicineIndent(1)`	• The zero line is not committed — a zero that got through would reach the pharmacy as an order for nothing. • The medicine is left on the form, and goes in on the second attempt — which is what says it was refused for the zero rather than for the drug.	AS EXPECTED The zero line was refused and the same medicine went in on the second attempt. Worth contrasting with DI-06: the diet form does not refuse it.	P2	PASS 14.7 s
TC_NS_010 line 271	The patient panel cannot be filled in by hand	Signed in; Nurse Station open.	1. Assert *Indent for*, Name, Cabin and Admitted DateTime are disabled. 2. Assert Item Description is disabled too.	—	• All five are disabled in the markup. The indent is bound to an admission, and a ward card is the only thing that binds it. • A typed-in name would file the indent against a patient the ward has no record of.	AS EXPECTED	P2	PASS 3.5 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
TC_NS_011 line 292	Verify Indent lists the ward's pending indents	Signed in; Nurse Station open. Skips if the ward has no indent pending.	1. Open the Verify Indent pane — its tabs and grid are not in the DOM while Add Indent is showing. 2. Assert the four status tabs and which one the pane opens on. 3. Read the grid's headers, then its rows.	Statuses `Pending`, `Partially Served`, `Served`, `Cancelled`; columns from `VERIFY_COLUMNS`	• All four status tabs are present and the pane opens on **Pending**. • Every named column is present, in order. • Assertions are made about the rows on screen, not the ward's whole backlog — the grid lists sixteen indents and stops, ordered by priority rather than by time.	AS EXPECTED	P2	PASS 8.3 s

17. Regression — Consultancy Service

Spec: `tests/regression/consultancy-service.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, the **Consultancy Service** tab — where the ward puts a consultant's visit, or any other chargeable service, onto an admitted patient's bill.

How the tab works, since nothing on it says. The ward list fills a patient panel of four read-only boxes. Under that is one line entry row — Service, Doctor Name, Service Date, Schedule, Rate, Qty — and *Enter on Qty* is what commits a line: there is no Add button, the only button between the two grids being Save. The upper grid is the **cart**, lines costed but not yet billed, with Total Amount and a discount box under it. The lower grid is the **bill**, every consultancy already saved against the admission, with a bin that opens a cancel-with-reason dialog.

Nine cases carry KNOWN FAIL **.** Each one asserts the behaviour the tab is supposed to have and is marked `test.fail()`, so a run reports it as expected-to-fail — and reports it as *unexpectedly passing* on the day the module is corrected, which is the signal to drop the marker and keep the assertion. Nothing is rewritten to match the defect, which would turn it into a documented feature and leave nothing watching for the fix. The nine are raised as **BUG-009** … **BUG-016** in the companion bug report.

What writes. Only CS-19 … CS-22 save. Each charges a live admission and takes its own charges back off again through the tab's cancel dialog, in a `finally`, so a failure mid-case still cleans up — a cancellation with a stated reason, recorded in IPD Bill Change History, not a deletion. Each prefers a service the catalogue prices at zero, so a run that dies between the save and the cleanup has still moved nobody's bill. The cleanup takes off *only* the rows its own case added: on a long-stay patient the bill is fifteen real consultancies, and a cleanup that emptied it would unpick a month of somebody's treatment.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-01 `line 100`	Consultancy Service opens from the Nurse Station tab strip, with both grids	Signed in; on the launcher.	1. Open the Hospital module and click Nurse Station. 2. Assert the page lands on Medicine Indent — Consultancy Service is four tabs along. 3. Open Consultancy Service and read the entry row and both grid headers.	Columns from `CART_COLUMNS` and `BILL_COLUMNS`	• Service, Doctor Name and Schedule lookups, Service Date, Rate and Qty are all on screen; Save is enabled. • Cart and bill each carry their own named columns, in order, plus a last one for the bin.	AS EXPECTED	P1	PASS 7.8 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-02 `line 136`	The ward list narrows to a bed or an admission number	Signed in; patients on the ward.	1. Count the whole ward list. 2. Search a live admission number; assert one card comes back and it is that admission. 3. Search the ward prefix off that bed; assert every card returned matches it. 4. Search a term the ward does not hold.	Admission number and bed read live off the list; `noSuchTerm` from `consultancyService()`	• An admission number is unique, so it brings back exactly one card. • A bed matches by substring — "ICU" reaches NICU-11 and Neuro ICU-13 — so every card returned contains the term rather than the count being fixed. • A term no bed matches empties the list rather than falling back to all of it.	AS EXPECTED	P2	PASS 12.8 s
CS-03 `line 181`	Picking a ward patient fills the patient panel, and none of it can be typed into	Signed in; a patient on the ward.	1. Pick a ward patient and read the panel back. 2. Assert all four patient fields are disabled.	Ward patient at the run's rotating ward offset	• *Inv.No* is the admission number off the card that was clicked — the tab names it Inv.No, but it is not an invoice number of its own. • Name is filled and Cabin No holds the card's bed. • All four are disabled: the ward list is the only way a patient gets onto this form. • *Not asserted:* that Assigned Doctor is filled. An admission can genuinely be under no consultant yet.	AS EXPECTED	P1	PASS 7.7 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-04 line 215	The Service lookup answers a search with the services it matches, and their rates	Signed in; a patient picked.	1. Search the Service lookup and read every row it offers. 2. Check each name against the term and each rate for readability. 3. Record any names the catalogue offers at more than one rate.	`serviceSearch` — one term matching a page of services	• Every service returned contains the search term; every rate reads as a number and is not below zero. • *Recorded, not asserted:* the catalogue holds entries that share a name and differ only in rate — two called "ICU Visit", at 1000 and at 0. That is the ward's data, and the lookup does show the rate beside each.	**AS EXPECTED** Every service returned contained the term and carried a readable rate. The run also recorded services the catalogue offers under one name at more than one rate — two called *ICU Visit*, at 1000 and at 0. That is the ward's data rather than a defect in the tab, and the lookup does show the rate beside each.	P2	**PASS** 11.1 s
CS-05 line 261	Picking a service puts its catalogue rate on the form, and Schedule offers two	Signed in; a patient picked. Skips if nothing matching the term carries a rate today.	1. Assert nothing is picked on arrival and the Rate box reads 0. 2. Pick the first priced service in the lookup; read the Rate box. 3. Pick a doctor through the Doctor Name lookup. 4. Open Schedule and read every row.	`serviceSearch`, `doctorSearch`; schedules from `SCHEDULES`	• The service picked is the one that lands on the form, and the Rate box takes the catalogue rate. • The Doctor lookup returns the consultant searched for. • Schedule holds exactly **On Time** and **Off Time** — no placeholder, no third option.	**AS EXPECTED**	P1	**PASS** 16.4 s
CS-06 line 297	A line with no service, or no doctor, is refused	Signed in; a patient picked. Test timeout raised to 180 s — each refusal costs the full wait for a line that never arrives.	1. Press Enter on Qty with nothing picked at all. 2. Pick a service, no doctor, and press Enter on Qty.	Qty 1 both times; expected text `MESSAGES.missingFields`	• Both are refused with *Please select fields.* — a quantity on its own is not a line. • The cart stays empty. This is the guard CS-17 then walks straight through with the lookup's blank row.	**AS EXPECTED**	P1	**PASS** 39.0 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-07 `line 328`	A quantity of zero, or a negative one, is refused	Signed in; a patient picked. Test timeout raised to 180 s.	1. Fill a complete line and commit it at a quantity of **0**. 2. Do the same at **−5**. 3. Read the service and doctor boxes back after the refusals.	Quantities 0 and −5; expected text `MESSAGES.badQuantity`	• Both are refused and named as quantity problems; neither reaches the cart. • The refusal leaves the service and the doctor where they were, so the nurse corrects the number rather than picking the service again.	AS EXPECTED	P1	PASS 1 m 32 s
CS-08 `line 364`	Enter on Qty adds the line at rate × qty, and Total Amount is the cart	Signed in; a patient picked.	1. For each quantity in the run's set: pick the next service in the lookup, add a doctor and a schedule, press Enter on Qty. 2. Read each cart line back against the catalogue entry it came from. 3. Sum the cart and compare against Total Amount. 4. Read the entry row after the last commit.	Quantities from `consultancyService()`, schedules alternating On Time / Off Time	• Each line carries the service picked, the quantity typed, the catalogue rate, and an amount of **rate × qty**. • The schedule reaches the line. • Total Amount equals the sum of the cart. • The entry row clears Qty and Doctor but keeps the service and its rate — the ward often bills the same service twice over.	AS EXPECTED	P1	PASS 1 m 4 s
CS-09 `line 410`	A cart line can be taken off again before it is saved	Signed in; a patient picked; two services in the catalogue to pick from.	1. Commit two different services at qty 1 each. 2. Assert Total Amount is the two amounts added. 3. Press the bin on the first row.	Two services off one lookup search	• The bin acts on the first click — no confirmation, no message; the row simply goes. • Exactly one row goes, and it is the right one. • Total Amount drops to the surviving line's amount.	AS EXPECTED	P2	PASS 1 m 3 s
CS-10 `line 450`	Save with an empty cart is refused	Signed in; a patient picked; cart empty.	1. Count the bill rows. 2. Press Save with nothing in the cart.	Expected text `MESSAGES.emptyCart`	• The save is refused and says so. • The bill is unchanged — an empty Save puts nothing on it.	AS EXPECTED	P1	PASS 9.9 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-11 `line 471`	The bill lists what the ward has already charged this admission	Signed in. Walks the first ten beds for a patient with a bill; skips if none has one. Test timeout raised to 180 s.	1. Step through the ward until a patient with bill rows turns up. 2. Read every row of the bill and check it against itself.	Live bill data — a patient with nothing charged yet is the normal case on a quiet ward	• Every charge names a service, is billed in a quantity above zero at a rate not below zero, and carries **amount = rate × qty − discount**. • Every charge names who served it and carries a date in `d/M/yyyy h:mm:ss AM` form. • *Recorded, not asserted:* the services on the bill with no doctor against them — CS-17 is how they get there.	**AS EXPECTED** Every charge was internally consistent (amount = rate × qty − discount), dated, and attributed to whoever served it. The run recorded the services already on the bill with **no doctor** against them — BUG-014 is how they got there.	P1	**PASS** 7.8 s
CS-12 `line 515`	The cart is emptied when another patient is picked	Signed in; two different patients on the ward. Test timeout raised to 180 s.	1. Pick the patient in bed A and commit one service to the cart. 2. Click the patient in bed B — the mis-click this case is about. 3. Read the patient panel, the cart and Total Amount.	Two consecutive ward patients; one service at qty 1	• Picking a patient starts that patient's bill: the cart is emptied and Total Amount returns to zero, because nothing in it belongs to the patient now on screen.	The patient panel moved to the second patient — name, bed and Inv.No all changed — and **the cart did not**. The first patient's line was still listed, and their money still in Total Amount, under the second patient's name. Save from there bills it to them, and nothing on screen says so. What would have been billed, and to whom, is recorded as a run annotation. Raised as **BUG-009**.	P1	**KNOWN FAIL** 54.3 s · BUG-009
CS-13 `line 568`	The cart is emptied when the tab is left and come back to	Signed in; a patient picked. Test timeout raised to 180 s.	1. Commit one service to the cart. 2. Go to Medicine Indent and back to Consultancy Service. 3. Read the cart once the pane has settled, then the patient panel.	One service at qty 1	• A cart with no patient against it should not exist: either both survive the round trip, or neither does.	The round trip *did* clear the patient panel and left the cart holding its line, with a figure under it in Total Amount, against nobody at all. Read after the pane settles, not the moment it paints: coming back repaints the grid empty for about a second before refilling it, so a quick look suggests the cart was cleared when it was not. Raised as **BUG-010**.	P1	**KNOWN FAIL** 41.3 s · BUG-010

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-14 `line 620`	The catalogue rate cannot be typed over on the form	Signed in; a patient picked. Skips if nothing matching the term carries a rate today.	1. Pick a priced service and read its catalogue rate. 2. Type the Rate box down to **1** and commit the line. 3. Read the rate and amount the cart line carries.	Rate typed *down* to 1, never up — the cart is never saved here, and a case that typed a rate up and then died before clearing it would leave a bigger number on screen than the ward meant to charge	• The Rate box is disabled, and the line goes in at the catalogue rate. A ward nurse adding a consultant's visit is not the person who prices it.	The box is editable and the line carried whatever was typed: a service the catalogue prices at 1000 committed at **1**, at amount 1. The catalogue rate and what the line carried are recorded as a run annotation. Raised as **BUG-011**; CS-22 follows it through the save.	P1	KNOWN FAIL 52.9 s · BUG-011
CS-15 `line 663`	The Service Date chosen survives committing the line	Signed in; a patient picked.	1. Read the date the box opened on — read back rather than compared against this machine's clock, which disagrees with the app either side of midnight. 2. Set the box to a back-date; pick a service, a doctor and a schedule. 3. Assert the back-date is still there right up to the Enter. 4. Commit the line and read the box once it has settled.	`backdated` from `consultancyService()`	• The date chosen survives the commit, so a service performed three days ago can be entered as three days ago.	The box took the date and kept it through the whole of the entry — right up to the Enter — then dropped it a render later, back to the day the tab opened on. The saved row carries the moment of the save rather than either. Raised as **BUG-012**.	P2	KNOWN FAIL 21.5 s · BUG-012
CS-16 `line 720`	Service Change comes off the amount	Signed in; a patient picked. Skips if nothing matching the term costs more than the discount.	1. Pick a service priced above the discount and fill the line. 2. Type the discount into the Service Change box under Total Amount. 3. Commit the line and read it back.	`discount` from `consultancyService()`	• The discount reaches the cart line's Service Change column and comes off the amount.	The box kept what was typed into it, so nothing on screen suggested it had been ignored — but the cart line carried no Service Change and the amount was the full rate. The saved row does not carry it either. The form has the box, the cart a column for it and the bill a Discount column, and nothing reads any of the three. Raised as **BUG-013**.	P2	KNOWN FAIL 19.1 s · BUG-013

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-17 line 769	The blank row in the Doctor Name lookup cannot stand in for a doctor	Signed in; a patient picked. Skips if the doctor list no longer opens on a blank row. Test timeout raised to 180 s.	1. Open the unsearched Doctor Name lookup and confirm the first row carries no code and no name. 2. Pick a service, then click that blank row. 3. Commit the line.	The lookup's own first row — a blank record in the consultant list	• Doctor Name is required and a blank record is not a doctor, so the commit is refused with *Please select fields.* — exactly as it is when the lookup is never opened at all (CS-06).	Having opened the lookup and clicked its blank row was enough to satisfy the guard. The line was committed with **no doctor against it**, and the same line saves — which is how the blanks CS-11 records reach live bills. Raised as **BUG-014**.	P1	KNOWN FAIL 47.9 s · BUG-014
CS-18 line 813	A lookup search that matches nothing says so	Signed in; a patient picked.	1. Search the Service lookup for a term no service matches. 2. Assert nothing selectable comes back. 3. Look for an empty-state message in the panel.	`noSuchTerm`	• Nothing selectable comes back, and the panel says there is no such service.	Nothing selectable came back, which is right. What the nurse is shown is an empty bordered row with no text in it — the same picture as a lookup that has not finished loading. Raised as **BUG-015**.	P3	KNOWN FAIL 27.9 s · BUG-015
CS-23 line 844	The bill lists every service charged, however many there are	Signed in. Walks the first eighteen beds for a patient already at the cap; skips if none is. Test timeout raised to 180 s.	1. Step through the ward until a patient's bill is at fifteen rows. 2. Search the tab for a pager, a page-size box or a total. 3. Count the rows the bill grid lists.	`BILL_ROW_CAP` = 15	• The bill lists every service charged against the admission, or says that it does not.	The grid stopped at **fifteen** rows, and the tab carried no pager, no page-size box and no total. A sixteenth consultancy is charged and then cannot be seen here — and because the bin lives on the row, it cannot be cancelled from this tab either. Which beds are at the cap is recorded as a run annotation. Raised as **BUG-016**.	P1	KNOWN FAIL 13.8 s · BUG-016
CS-19 line 909	Save puts the cart on the bill and clears it WRITES	Signed in; a patient with room below the bill cap; a free service in the catalogue. Test timeout raised to 240 s.	1. Baseline the bill, then commit one free service to the cart. 2. Press Save. 3. Read the cart, the bill count and the new row. 4. **Cleanup:** cancel off only the rows the baseline cannot account for.	A service the catalogue prices at zero, qty 1	• The save is acknowledged; the cart is emptied — the one place the tab does clear it. • The bill grows by exactly one row, appended last, carrying the same service, quantity, rate and amount as the cart line, and naming who charged it. • Everything that was on the bill before is still on it, unchanged.	AS EXPECTED The save was acknowledged, the cart emptied, and exactly one row appended to the bill carrying the cart line's service, quantity, rate and amount. The rest of the bill was untouched. The case cancelled its own row back off afterwards; how many it removed is recorded as a run annotation.	P1	PASS 46.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
CS-20 line 970	A saved service is cancelled with a reason, and the rest of the bill is left alone WRITES	Signed in; a patient with room for two below the cap; a free service. Test timeout raised to 240 s.	1. Save two lines of the same free service at **different quantities**, so the two can be told apart — a Save stamps every row it writes with the same moment, so ServiceDate separates nothing. 2. Open the cancel dialog on the first of them; read what it names and which reason chips it offers. 3. Press *Keep service* and assert nothing was cancelled. 4. Cancel it properly, with a reason. 5. **Cleanup:** cancel off what is left of this case's own rows.	Two lines at qty 1 and qty 2; reasons from `CANCEL_REASONS`	• The dialog names the service it is about to take off and offers every reason chip. • *Keep service* leaves the bill at two. • The cancellation is acknowledged, takes off exactly one row, and it is the right one — the other survives. • The rest of the bill is untouched.	AS EXPECTED The dialog named the service and offered every reason chip; *Keep service* cancelled nothing; the cancellation took off exactly one row and left the other standing.	P1	PASS 1 m 41 s
CS-21 line 1046	The cancel dialog refuses a blank reason WRITES	Signed in; a patient with room below the cap; a free service. Test timeout raised to 240 s.	1. Save one free service, then open the cancel dialog on it. 2. Confirm with the reason box empty. 3. Fill a single character and confirm again. 4. **Cleanup:** close the dialog, then cancel off this case's own row.	Reasons `''` and `x`	• A blank reason is refused inline — a message under the box, not a snackbar — and the dialog stays open with the service still on the bill behind it. • A one-character reason is refused the same way. • The bill count never moves.	AS EXPECTED Both the blank reason and the one-character reason were refused inline, the dialog stayed open, and the bill count never moved.	P1	PASS 50.9 s
CS-22 line 1108	A rate typed over the catalogue rate does not reach the bill WRITES	Signed in; a patient with room below the cap; a *priced* service, because the whole of this case is about what happens to the price. Test timeout raised to 240 s.	1. Pick a priced service and read its catalogue rate. 2. Type the Rate box down to **1**, commit, and Save. 3. Read the rate on the saved bill row. 4. **Cleanup:** cancel off this case's own row.	Rate typed down to 1, never up — this is a live bill, and the finding is the same either way round	• The bill carries the catalogue rate, whatever was typed at the nurse station.	The saved bill row carried the rate typed at the nurse station, not the catalogue rate. The patient, the catalogue rate and what was billed are recorded as a run annotation. CS-14 is the same finding at the form; this is it on the bill, which is where it matters. Raised as **BUG-011**.	P1	KNOWN FAIL 46.0 s · BUG-011

18. Regression — Diet Indent

Spec: `tests/regression/diet-indent.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, the **Diet Indent** tab — where the ward nurse orders meals for an admitted patient: pick the patient off the ward list, set an outlet and a priority, list the food patterns, save. Medicine Indent, the tab next to it, is section 16; the two share a route and almost no markup, which is why they have a page object each.

Three things worth knowing before reading the cases. (1) A patient cannot be given the same food pattern twice while the first order is still pending — the line is turned away at Enter with *Already has indent for this food pattern*. That is the duplicate guard this module has, and DI-14 is the case about it. It is also why no case names a food pattern: each asks for the first one the ward will still take, because every save leaves the next case one pattern less to choose from. (2) **The quantity is not recorded.** Every line lands in the indent table as 1, whether 5, 9 or 0 is typed, and reaches Verify Indent as 1 as well — DI-06 is the case that says so, and it is why every other case reads quantities back rather than assuming them. (3) Nothing is answered inline: every refusal and every save arrives as a Blazorise snackbar, in the same element with the same classes, so the text is the only thing separating them.

DI-04, DI-05, DI-09 and DI-12 … DI-15 write. Each files at least one real diet indent against a real patient on a live ward; DI-15 files two. They land as Pending on Verify Indent, where the canteen sees them and where they can be cancelled, and each carries a mark unique to the case in its Remarks — both how the case finds its own indent afterwards and how whoever clears them down can tell them from a nurse's. Retries are off for that describe: a retry would file a second indent, and DI-05, DI-13 and DI-14 are counting indents.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DI-01 `line 47`	Diet Indent > Add Indent opens from the Hospital module menu	Signed in; on the launcher.	1. Open the Hospital module and click Nurse Station. 2. Assert the page lands on Medicine Indent, then open Diet Indent. 3. Read the form, both dropdowns and the item table's headers.	Three priorities and the outlets from `DietIndentPage`; columns from `ITEM_COLUMNS`	• The page opens on Medicine Indent — Diet Indent is a click away, which is the step this case proves. • Outlet, the pattern lookup, Remarks and the item table are all on screen. • Priority offers *Select Item* plus three entries — not the Medicine Indent list of six, which is the sharpest evidence on the page that the two tabs are separate forms. • Outlet offers *Select Canteen* plus its canteens; the item table's fourth column is unlabelled and holds the delete button.	AS EXPECTED	P1	PASS 8.1 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DI-02 line 89	Selecting a ward patient fills the indent form with that patient	Signed in; Add Indent open; a patient on the ward.	1. Assert the four patient boxes are empty and disabled before anything is clicked. 2. Click the ward card at the run's ward offset. 3. Read the patient panel back against the card.	Ward patient at the run's rotating ward offset; nothing is indented	• The boxes are disabled, so the ward card is the only thing that can fill them. • *Indent for* carries the patient's UHID, not the admission number the card was found by. • Name and admission date arrive; the form prints the bed as `P(HCU-27)` where the card prints `HCU-27`, so the card's bed is contained in the form's.	AS EXPECTED	P1	PASS 4.8 s
DI-03 line 116	Save without a priority is refused	Signed in; Add Indent open; a patient on the ward.	1. Select a patient, choose an outlet, add one food pattern. 2. Leave Priority unset and press Save.	Everything but the priority, so the refusal can only be about the priority	• Refused with `MESSAGES.noPriority`. • Refused, not saved: the line and the patient are still on the form — a save empties it.	AS EXPECTED	P1	PASS 15.6 s
DI-06 line 135	A quantity of zero should not be added to the indent	Signed in; Add Indent open; a patient on the ward.	1. Select a patient. 2. Offer a food pattern at a quantity of **0**, stepping over any pattern the ward already has on order so the verdict is about the quantity.	Quantity 0	• The line is refused, and the refusal names the quantity — the way Medicine Indent refuses it (TC_NS_009).	A quantity of 0 was taken like any other and the line was added — **reading 1**. The quantity is not recorded at all: every line lands as 1 whatever is typed, and reaches Verify Indent as 1. Raised as **BUG-017**.	P1	KNOWN FAIL 16.1 s · BUG-017

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DI-07 line 168	A negative quantity is not accepted	Signed in; Add Indent open; a patient on the ward; a pattern on the form.	1. Set the quantity box to −5 and read it back. 2. Clear it and *type* −5 keystroke by keystroke; read it back.	Set and typed, because they are different paths into the control: fill writes the value, pressSequentially sends the keystrokes a nurse would, and a box can refuse one and take the other	• Neither path leaves a negative value in the box. • No minus sign survives typing.	AS EXPECTED	P2	PASS 5.5 s
DI-08 line 198	Save with the required fields empty is refused, one reason at a time	Signed in; Add Indent open.	1. Press Save on a form with nothing on it. 2. Add a patient and a priority, still nothing to send up; press Save again. 3. Press Enter on Quantity with no pattern picked.	Expected texts `MESSAGES.noItems` and `MESSAGES.noPattern`	• The empty form is refused for the indent table, and so is the form with a patient but no food pattern. • A line with no pattern is refused *at the line*, before Save is ever reached — Enter on Quantity is the add action. • Nothing reaches the item table.	AS EXPECTED	P1	PASS 15.5 s
DI-08b line 225	Save with an indent but no patient is refused	Signed in; Add Indent open.	1. Choose a priority and an outlet and add one food pattern — no patient. 2. Press Save.	Expected text `MESSAGES.noPatient`	• Refused, and the app names the patient precisely — the patient panel is the one required field a nurse cannot type into, so leaving it empty is the easiest mistake to make on this form. • The line stays on the form.	AS EXPECTED	P1	PASS 10.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DI-10 line 240	Several food patterns can be listed on one indent	Signed in; Add Indent open; a patient on the ward.	1. Select a patient and a priority. 2. Add three lines, each its own food pattern with its own remarks. 3. Read the item table back.	Three lines from `dietIndent(3)`, each with remarks `<mark> line n`	• Three lines, three **different** patterns, in the order they were added, each carrying its own remarks. • *Deliberately not asserted:* that the quantities are the ones typed. They are not — the table reads 1 on every line. Recorded as an annotation so the report carries the evidence; DI-06 is the case that asserts it.	**AS EXPECTED** for the patterns and the remarks: three lines, three different patterns, in the order added, each with its own remarks. The quantities were **not** the ones typed — the table read 1 on every line. Recorded as a run annotation rather than asserted here; DI-06 is the case that asserts it. **BUG-017**.	P2	**PASS** 28.0 s
DI-11 line 276	Letters and symbols are not accepted as a quantity	Signed in; Add Indent open; a patient on the ward; a pattern on the form.	1. Type `abc`, then `!@#`, then `1a2` into the quantity box, keystroke by keystroke. 2. Read the box back after each.	Three rubbish values	• The box holds digits and nothing else, or nothing at all — anything it took from `abc` or `!@#` would show up as a character the pattern does not allow.	**AS EXPECTED**	P2	**PASS** 6.8 s
DI-04 line 339	A diet indent saves against an admitted patient **WRITES**	Signed in; a patient on the ward. Test timeout raised to 240 s; retries off.	1. Build a one-line indent: patient, outlet, priority, one food pattern. 2. Press Save.	One line from `dietIndent()`, remarks carrying the case's own mark	• Save answers `MESSAGES.saved`. • The form empties — the message alone is not proof, since the snackbar carries refusals too, in the same element.	**AS EXPECTED**	P1	**PASS** 10.3 s
DI-05 line 354	Clicking Save twice files the indent once **WRITES**	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save. 2. Press Save twice more with nothing touched in between — the double-click a nurse makes when the host is slow. 3. Find the indent on Verify Indent by its mark and count how many carry it.	One line; the case's own Remarks mark	• The first Save is acknowledged; the second and third are refused for an empty indent table. • **Exactly one** indent carries the mark. The form emptying is a weaker guard than a disabled button — it depends on the clear landing before the second click does — so the count on Verify Indent is what this case stands on.	**AS EXPECTED**	P1	**PASS** 29.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DI-09 line 378	The remarks and quantity entered reach the saved indent WRITES	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save. 2. Find it on Verify Indent by its mark and read the row.	One line; remarks are the only field a case can set and read back exactly	• Remarks survive the save intact. • The saved indent is for the food pattern that was ordered. • The quantity on Verify Indent agrees with the indent table — which is the same 1 for every line. Checked for consistency between the two screens, not against what was typed; DI-06 is the case about that.	AS EXPECTED Remarks survived the save intact and the food pattern matched. The quantity agreed between the indent table and Verify Indent — both 1 — which is what this case checks. That it is 1 rather than what was typed is BUG-017, not this case.	P2	PASS 20.3 s
DI-12 line 403	A saved indent appears on Verify Indent as Pending WRITES	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save. 2. Find it on Verify Indent by its mark and read the whole row.	One line; outlet and priority set from `dietIndent()`	• The indent carries a number of 10+ digits, the priority and the outlet it was filed under. • The grid prints the bare bed where the form printed `P(Ex-HDU-02)`. • Status is **Pending** — waiting for the canteen — and it is filed under a user.	AS EXPECTED	P1	PASS 14.6 s
DI-13 line 422	A saved indent survives a reload, and is not filed twice by it WRITES	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save; note its number. 2. **Reload the browser** — the nurse who is not sure the save took. 3. Read the form, then count indents carrying the mark.	A full reload, not a tab switch — the risk it covers is a form that re-posts what it still holds	• The reloaded form holds no line and no patient. • Exactly one indent carries the mark, under the same number as before, still Pending.	AS EXPECTED	P1	PASS 24.5 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DI-14 `line 447`	Re-saving the same patient indent does not file it again WRITES	Signed in; a patient on the ward. Test timeout raised to 360 s; retries off.	1. File a one-line indent for a patient and Save. 2. Go back to the form, pick the *same* patient and offer the *same* food pattern. 3. Press Save with nothing in the table. 4. Count indents carrying the mark.	Expected text `MESSAGES.duplicatePattern`	• The repeat line is turned away *at Enter*, before Save is ever reached — that patient already has that food pattern on order. • With nothing in the table, Save has nothing to file either. • Still exactly one indent, under the original number.	AS EXPECTED The repeat line was turned away at Enter with *Already has indent for this food pattern*, before Save was reached, and only one indent carried the mark.	P1	PASS 36.6 s
DI-15 `line 484`	Two patients get an indent each WRITES	Signed in; two different patients on the ward. Test timeout raised to 420 s; retries off. **Files two indents.**	1. Read two different ward patients. 2. File and save an indent for each, back to back, without reloading between them. 3. Find both on Verify Indent by their marks.	Two independent `dietIndent()` sets, each with its own mark	• Both saves are acknowledged and the form clears between them — one nurse can work down the ward without the form carrying anything over. • Two indents, two **different** numbers, two beds — not one indent that the second save overwrote.	AS EXPECTED	P1	PASS 44.6 s

19. Regression — Diet Dashboard

Spec: `tests/regression/diet-dashboard.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, the **Diet Dashboard** tab — where the ward sees every diet indent raised in a date range and starts or stops the meal a patient is actually on. This suite reads what section 18 writes.

What the two grids are, since nothing on the page says. They carry identical headers, no caption and no heading; only the button in the last cell differs. The upper grid is the diet each patient is currently on — one row per admission number, never more — and its button is **Stop**. The lower grid is everything else in the range, and its button is **Continue**. Status does not separate them: Pending, PR Submitted, Served and Reviewed rows turn up in both. DD-04 pins that down, and DD-12 is the case about it not being written anywhere on screen.

Run date note. Every case reads the live ward. A day the ward has not ordered on yet leaves the dashboard empty, which is the right answer and not a failure, so the cases that need rows skip rather than fail when there are none. DD-03, DD-04 and DD-08 skipped in the 05 Oct run and again on 10 Oct — on 05 Oct because **BUG-018** had the dashboard filtered to a day with nothing on it, on 10 Oct because the run was at 01:30 and the ward had genuinely not ordered yet.

DD-18 is not a Diet Dashboard case. It measures the ERP host's clock, which stamps every date in the system, and it lives in this suite because this is the tab where the skew shows — and because DD-02 and DD-13 used to carry the assertion and failed for half of every day over it. See the fix-pass note under section 4.

DD-14 … DD-17 write. Each files its own diet indent through the Diet Indent tab first and drives Stop and Continue against that one, so the suite never stops a meal the ward ordered, and each leaves the ward as it found it. Retries are off for that describe: a retry would file a second indent, and these cases are counting rows.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-01 `line 110`	Diet Dashboard opens from the Nurse Station tab strip, with both grids	Signed in; on the launcher.	1. Open the Hospital module and click Nurse Station. 2. Assert the page lands on Medicine Indent — Diet Dashboard is two tabs along. 3. Open Diet Dashboard; read the filter bar and both grids' headers. 4. Read each grid's action buttons, where it has rows.	Nine columns from `COLUMNS`; buttons from `ACTIONS`	• Start Date, End Date, Search and Show are all on screen. • Both grids carry the same nine named columns plus a tenth for the button. • The upper grid offers only Stop and the lower only Continue. A quiet ward can leave either empty, so each is checked only when it has rows.	AS EXPECTED	P1	PASS 10.3 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-02 line 153	The dashboard opens filtered to today, and says so in the date boxes	Signed in; Diet Dashboard open.	1. Read the hidden Start Date and End Date inputs the app posts. 2. Read every row and check its Indent Date against today. 3. Read the date boxes the nurse actually sees.	The day is read off the dashboard's own hidden Start Date input, not off the test host's clock — see the Actual column	• The filter is a **single day** — both hidden inputs carry the same date — and every row on screen was raised on it. • The boxes on screen show the range in force.	**Rewritten 10 Oct 2026.** It used to assert that the app's date equalled `daysAgo(0)` from the test host, and failed on 05 Oct with `2026-10-06` against a client date of `2026-10-05`. That turned out to be the host clock running ~12 h fast (**BUG-018**), not a Diet Dashboard defect — and it made this case fail for half of every day over something it is not about. The clock assertion now lives in `DD-18`; this case reads the day off the app and asserts what the tab owns. The expected-to-fail marker still covers the second half: the date boxes show nothing but placeholder text — **BUG-019**.	P1	KNOWN FAIL 22.8 s · BUG-019
DD-03 line 216	Every row carries a complete diet indent record	Signed in; Diet Dashboard open. Skips when the ward has not ordered a meal yet.	1. Read both grids. 2. Check every field of every row.	Statuses from `STATUSES`	• Every row carries an indent number of 10+ digits, an admission number of 8+, a patient, a bed and a diet. • Indent Date reads `dd/MM/yyyy` and Indent Time `hh:mm AM/PM`. • Status is one this suite knows. Remarks are not checked for content — they are the nurse's free text and are very often empty, which is not a defect.	**Skipped** — *the ward has not ordered a meal yet today*. Both grids were empty, so there was no row to read. The dashboard was filtered to a day nothing had been ordered on, which is **BUG-018** downstream rather than a quiet ward.	P1	SKIPPED 7.1 s · no rows (BUG-018)

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-04 line 248	A patient is on one diet at a time, and no indent is in both grids	Signed in; Diet Dashboard open. Skips when no patient is on a diet today.	1. Read both grids. 2. Look for an admission number appearing twice in the running grid. 3. Look for an indent number in both grids at once.	Live ward data	• No patient is shown running two diets at the same time — this is the contract the upper grid keeps, and the reason the tab can offer a single Stop per patient. • No indent is in both lists; that would leave the nurse two contradictory buttons for one order. • *Deliberately not asserted:* that status splits the two grids. It does not — the same status appears on both sides. Recorded as an annotation.	**Skipped** — *no patient on the ward is on a diet today*. Same cause as DD-03: **BUG-018** left the running grid empty.	P1	SKIPPED 7.3 s · no rows (BUG-018)
DD-05 line 284	The date range filters both grids	Signed in; Diet Dashboard open. Test timeout raised to 180 s.	1. Filter to the last week and read both grids. 2. Check every row's Indent Date falls inside the window. 3. Filter to a window a month in the future.	`window` = last 6 days; `future` = 30–37 days ahead	• A week of a live ward brings back indents, and every one of them falls inside the window, in both grids. • A window no indent can fall in empties both grids rather than falling back to today.	AS EXPECTED A week brought back indents, every one inside the window, in both grids; a window a month ahead emptied both rather than falling back to today.	P1	PASS 18.1 s
DD-06 line 319	A date range that runs backwards is refused, not silently emptied	Signed in; Diet Dashboard open. Test timeout raised to 180 s.	1. Set the same week as DD-05 with its ends swapped — the range a nurse produces by setting the End Date first. 2. Assert both grids came back empty. 3. Look for any complaint on the page.	`reversed` = today … 6 days ago	• A range the app cannot honour is refused — in the field or on screen — rather than answered with an empty ward.	Both grids came back empty and nothing on the page said why. That is also what a genuinely empty week looks like, what a search with no matches looks like (DD-09), and what BUG-018's wrong default looks like. Four situations, one picture. Raised as **BUG-020**.	P2	KNOWN FAIL 26.4 s · BUG-020

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-07 line 352	A wide date range is not silently cut off at 120 rows	Signed in; Diet Dashboard open. Skips if the ward raised fewer than 120 indents in the fortnight. Test timeout raised to 300 s.	1. Filter to the last fortnight; read both grids and check whether either is at the cap. 2. Filter to twice that window and read both again. 3. Compare the oldest indent in each grid across the two windows.	`ROW_CAP` = 120; windows of 14 and 28 days	• Doubling the window brings back something older. A ward busy enough to fill a grid over a fortnight was busy over the fortnight before it too.	Both windows came back with exactly **120 rows a grid** and the same oldest indent in each: the list is cut at 120 and the rest is not there. No pager, no page-size box, no total anywhere on the tab. The row counts for both windows are recorded as a run annotation. Raised as **BUG-021**.	P1	KNOWN FAIL 18.1 s · BUG-021
DD-08 line 398	Search filters both grids, on any column and in any case	Signed in; Diet Dashboard open. Skips when no diet indent was raised today to search for. Test timeout raised to 180 s.	1. Read both grids and take a seed row off whichever has one. 2. Search its indent number, then its patient name in two cases, then its bed. 3. Clear the box.	Seed row read live — which beds are occupied changes by the hour	• An indent number narrows the whole dashboard to that one row — the sharpest evidence that the box filters both grids at once. • A patient name matches in upper and lower case, and every row returned is that patient's. • A bed is a different column again, and matches. • Clearing the box puts the dashboard back the way it was.	**Skipped** — *no diet indent was raised today to search for*. There was no seed row to search on, for the same reason as DD-03: **BUG-018**.	P1	SKIPPED 7.1 s · no rows (BUG-018)
DD-09 line 451	A search that matches nothing says so	Signed in; Diet Dashboard open.	1. Search a term no patient can match. 2. Assert both grids are empty. 3. Look for an empty-state message.	`ZZ-NO-SUCH-PATIENT-ZZ`	• An empty result says it is empty.	Both grids emptied to bare headers with nothing said — so a nurse cannot tell a filter that matched nothing from one the app would not run. Raised as **BUG-020**.	P2	KNOWN FAIL 25.3 s · BUG-020

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-10 `line 469`	Each grid sorts on its own, without disturbing the other	Signed in; Diet Dashboard open, filtered to the last week. Skips if either grid has fewer than two rows. Test timeout raised to 180 s.	1. Note the stopped grid's order. 2. Sort the running grid by patient name; read that column back. 3. Re-read the stopped grid's order. 4. Click the same header again.	Order is asserted as a *direction* rather than as the reverse of the first list: the grid cuts at 120 rows after sorting, so ascending hands back the first 120 names and descending the last 120	• The running grid goes into name order. • The stopped grid is exactly as it was — each grid carries its own sort. • A second click turns the same column round.	AS EXPECTED	P2	PASS 20.3 s
DD-12b `line 512`	The header says which column the grid is sorted by, and which way	Signed in; Diet Dashboard open, filtered to the last week. Skips if the running grid has fewer than two rows. Test timeout raised to 180 s.	1. Read the sort marks before anything is clicked. 2. Sort by Indent Date; read the marks on that column and on three others. 3. Click again and re-read. 4. Read the dates under the header.	Columns `indentNo`, `indentDate`, `patientName`, `status`	• Nothing is marked before a column is picked — the list arrives in the app's own order, which it does not claim to be any column's. • The caret goes on the sorted column alone, so it says *which* column the order is on and not merely that the grid is sorted. • It turns over with the order, and the mark agrees with the rows under it.	AS EXPECTED	P2	PASS 19.6 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-11 line 566	Indent Date and Indent Time sort chronologically, not as text	Signed in; Diet Dashboard open, filtered to the last week. Skips if the running grid has fewer than two rows. Test timeout raised to 180 s.	1. Sort by Indent Date and parse the column as dates. 2. Sort by Indent Time and parse the column as minutes past midnight.	A week rather than today: a single day is all one date, and a ward's quiet hours are where AM and PM meet	• Indent Date is in date order — `dd/MM/yyyy` sorted as text would put the 1st of a month before the 28th of the one before it. • Indent Time is in time order — sorted as text, every PM would come before every AM of the same hour. • Which meridiems the rows covered is recorded as an annotation.	AS EXPECTED Both columns sorted chronologically rather than as text. How many rows and which meridiems the run covered is recorded as an annotation — the AM/PM mix is what makes the time assertion meaningful.	P2	PASS 20.2 s
DD-12 line 612	Each grid is labelled, so a nurse can tell which list a row is in	Signed in; Diet Dashboard open.	1. For each grid, look for a caption, an `aria-label` or an `aria-labelledby`.	—	• Each grid is named — by a caption, a heading over the card, or at minimum an accessible label.	Neither grid carries a caption, an `aria-label` or an `aria-labelledby`. The two are rendered identically — same classes, same nine headers — and the button in the last cell is the only thing telling them apart. Raised as **BUG-022**.	P2	KNOWN FAIL 7.5 s · BUG-022
DD-13 line 633	Leaving the tab and coming back puts the dashboard back on today	Signed in; Diet Dashboard open. Test timeout raised to 180 s.	1. Widen the range to a week and type something into Search. 2. Go to the Diet Indent tab and back. 3. Read the search box, the hidden date inputs and every row.	Week window and a one-character search; the day the dashboard opened on, captured before anything is changed	• The tab refetches from scratch: the search box is empty again and the range is back to **the day the dashboard opened on**, with every row on screen raised on it.	AS EXPECTED — the search box clears and the range goes back to the opening day. **Rewritten 10 Oct 2026** for the reason DD-02 gives: it used to compare the range against the test host's clock and so failed on 05 Oct for **BUG-018**, which is the host clock, not this tab. It now pins the round trip against the day the dashboard itself opened on — which is what makes the case worth having, the alternative being a stale list from the last visit that would let a nurse act on a diet no longer running.	P1	PASS 22.8 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-18 line 675	The ERP host and the ward keep the same clock	Signed in. Not really a Diet Dashboard case — it lives here because this is the tab where the defect shows.	1. Send one `HEAD` request to the host and read its HTTP `Date` header — the host's own clock, in GMT, by definition. 2. Compare it against this machine's clock and record the difference in minutes. 3. Open the dashboard and record the day it opened on, for the annotation.	Tolerance of 10 minutes — wide enough that ordinary drift and the round trip never trip it	The host keeps the same clock as the ward it serves.	AS EXPECTED since 22 Oct 2026 — **BUG-018 fixed**, the skew now reads `+0`. It was **+718 minutes** on three consecutive readings on 10 Oct 2026: almost exactly ±12 h, the signature of a time-zone sign error (`UTC+6` configured as `UTC−6`) rather than of a drifting clock. The marker came off on the run that reported this case as unexpectedly passing; the assertion stays as a live guard, because a clock is the kind of thing that is corrected once and comes back after a reboot. Measured off the header rather than off a date on screen on purpose: the header is wrong at *every* hour, where the day-ahead symptom only appears from local noon. A case that compared dates would pass all morning and fail all afternoon, which is what DD-02 and DD-13 used to do.	P1	PASS 9.3 s
DD-14 line 768	An indent saved on Diet Indent appears as that patient's running diet WRITES	Signed in; a patient on the ward. Test timeout raised to 360 s; retries off. **Files one indent.**	1. File a diet indent through the Diet Indent tab and note its number. 2. Cross to the dashboard and search that number. 3. Read the row it lands in.	One line from `dietIndent()`, with the case's own Remarks mark	• The indent is in the **running** grid, against the right patient and the right bed. • It carries the food pattern ordered and the remarks typed. • Status is Pending and the action offered is Stop. • It is in one list, not both.	AS EXPECTED	P1	PASS 21.6 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-15 `line 805`	Stop takes the patient off the diet, and Continue puts them back WRITES	Signed in; a patient on the ward. Test timeout raised to 420 s; retries off. **Files one indent, then acts on that one only.**	1. File an indent and find it in the running grid. 2. Press **Stop**; read where the row went and what status it took. 3. Press **Continue**; read it back.	The case's own indent — a suite that reached for whatever row was at the top would be stopping meals a nurse ordered	• Stop moves the row to the stopped grid and rewrites the status to **PR Submitted**, leaving the diet itself alone; the running grid no longer shows it. • Continue closes the round trip exactly: same indent, same diet, same remarks, back in Pending and back in the running grid — which is also what puts the ward back as it was found. • *Recorded, not asserted:* neither button said anything. The row moving between two unlabelled grids is the entire feedback. DD-16 is the case about that.	AS EXPECTED Stop moved the row to the stopped grid and rewrote the status to **PR Submitted**, leaving the diet itself alone; Continue closed the round trip exactly and put the ward back. Neither button said anything — the row moving between two unlabelled grids was the entire feedback. Recorded as a run annotation; DD-16 is the case that asserts it.	P1	PASS 24.6 s
DD-16 `line 853`	Stop asks before taking a patient off the diet they are on WRITES	Signed in; a patient on the ward. Test timeout raised to 420 s; retries off. **Files one indent.**	1. File an indent and find it in the running grid. 2. Press Stop, watching for a dialog and for a message from the moment the button is pressed. 3. **Put the ward back** with Continue — before reporting, because the assertion below fails on purpose and a failing test runs no further.	The case's own indent	• Stopping a diet is clinical, and is asked about — or at the very least acknowledged afterwards.	Stop acted on the first click. No dialog, no message: by the time anything could be read, the row had moved and the status was written. The ward was put back with Continue before the case reported. Raised as **BUG-023**.	P1	KNOWN FAIL 23.8 s · BUG-023

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
DD-17 `line 888`	Continue on an older indent warns that the running diet will be displaced WRITES	Signed in; a patient on the ward. Test timeout raised to 540 s; retries off. **Files two indents for one patient.**	1. File two indents for the same patient, so the ward has an order to go back to. 2. Confirm the newer is running and the older has been superseded. 3. Press **Continue** on the older one and read where the newer went. 4. **Restore the ward:** Continue the newer indent again.	Two independent `dietIndent()` sets against one admission	• Continue says that it is about to displace whatever the patient is on now.	Continue on the older indent silently dropped the newer one out of the running grid — correctly, since a patient is on one diet at a time (DD-04), but with nothing on screen saying a live order had just been cancelled to do it. The ward was restored before the case reported. Raised as **BUG-023**.	P1	KNOWN FAIL 42.3 s · BUG-023

20. Regression — IPD Service Entry

Spec: `tests/regression/ipd-service-entry.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, the **IPD Service Entry** tab — where the ward puts a chargeable service, a procedure or an OT item onto an admitted patient's bill.

How the tab works, since nothing on it says. The ward list fills a patient panel of six read-only boxes — Inv.No, Name, Gender, Age, Assigned Doctor, CabinNo. Under that is one line entry row — Service, Code, Remarks, Service Date, Start Date Time, End Date Time, Rate, Qty — and *Enter on Qty* is what commits a line: there is no Add button, the only buttons under the cart being Save and Print All. The upper grid is the **cart**, lines costed but not yet billed, with Total Amount and a discount box under it. The lower grid is the **bill**, every service already saved against the admission, with who saved it and when.

Nothing in this suite saves, and that is itself a finding. Consultancy Service puts a bin on every row of its bill, which opens a cancel-with-reason dialog — which is how the cases over there can charge a live admission and take the charge back off again. This tab's bill grid has nine columns and no control of any kind on the row: a service saved here cannot be cancelled, corrected or removed from this tab at all. That is `IS-24` / **BUG-027**, and it is also why every case below stops at the cart. A case that pressed Save would be making a permanent change to a real patient's bill with no way to undo it. The one Save the suite does press is `IS-25`, against an empty cart, which writes nothing and is asserted to be empty first.

Fourteen cases carry KNOWN FAIL. Each asserts the behaviour the tab is supposed to have and is marked `test.fail()`, so a run reports it as expected-to-fail — and reports it as *unexpectedly passing* on the day the module is corrected, which is the signal to drop the marker and keep the assertion. Nothing is rewritten to match the defect, which would turn it into a documented feature and leave nothing watching for the fix. The fourteen are raised as **BUG-024** … **BUG-037** in the companion bug report.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-01 `line 119`	IPD Service Entry opens from the Nurse Station tab strip, with both grids	Signed in; on the launcher.	1. Open the Hospital module and click Nurse Station. 2. Assert the page lands on Medicine Indent — IPD Service Entry is five tabs along. 3. Open IPD Service Entry and read the entry row and both grid headers.	Columns from `CART_COLUMNS` and `BILL_COLUMNS`	• Service lookup, Code, Remarks, the three date boxes, Rate and Qty are on screen; Save is enabled and Print All is present. • Cart and bill each carry their own named columns, in order; the cart has a tenth for the bin and the bill does not.	AS EXPECTED	PASS 9.0 s	

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-02 line 159	The ward list narrows to a bed or an admission number	Signed in; patients on the ward.	1. Count the whole ward list. 2. Search a live admission number; assert one card comes back and it is that admission. 3. Search the ward prefix off that bed; assert every card returned matches it. 4. Search a term the ward does not hold.	Admission number and bed read live off the list; `noSuchTerm` from `ipdServiceEntry()`	• An admission number brings back exactly one card. • A ward prefix brings back only beds whose name contains it. • A term no bed matches empties the list rather than falling back to all of it.	AS EXPECTED	PASS 19.8 s	
IS-03 line 204	Picking a ward patient fills the patient panel, and none of it can be typed into	Signed in; IPD Service Entry open.	1. Assert Inv.No reads 0 before any card is clicked. 2. Click a ward card and read all six panel boxes back. 3. Assert each of the six is disabled in the markup.	A bed read live off the list, rotated per run by `wardIndex`	• Inv.No is the admission number of the card clicked; Name, Gender and Age are filled; CabinNo holds the bed. • All six boxes are disabled — the ward list is the only way to set a patient.	AS EXPECTED	PASS 16.6 s	
IS-04 line 253	The Service lookup answers a search with the services it matches, and their rates	Signed in; a patient picked.	1. Open the Service lookup and search a term the catalogue holds. 2. Read every row back as a name and a rate. 3. Assert each name contains the term and each rate is a readable number ≥ 0.	`serviceSearch` = "Visit"	• Every row returned matches the term. • Every row carries a rate, which is what tells two entries of the same name apart.	AS EXPECTED Flaked once on the first full run — a search click swallowed before Blazor wired it left the unfiltered catalogue on screen. The page object now re-sends a search whose grid did not move; it has not recurred.	PASS 16.0 s	
IS-05 line 288	Picking a service puts its catalogue rate on the form	Signed in; a patient picked. Skips if nothing matching the term carries a rate today.	1. Assert nothing is picked and Rate and Qty both read 0 on arrival. 2. Search the catalogue and take the first priced entry. 3. Read the Rate box and the Service Date box back.	`serviceSearch` = "Visit"	• The lookup puts the service that was clicked on the form, and its catalogue rate in the Rate box. • Service Date opens on a date *and a time* — the shape IS-17 goes on to test.	AS EXPECTED	PASS 19.9 s	

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-06 line 321	Qty refuses zero and negatives, and says why	Signed in; a free service picked.	1. Pick a service the catalogue prices at zero. 2. Press Enter on Qty with 0, then with −1. 3. Assert neither reached the cart, and read the message each raised.	Qty 0 and −1	• Both are refused with `Please enter a valid Qty.` — the wording until 22 Oct 2026 was `Please input correct Qty.` • The cart is still empty, and the service is left on the form so the nurse can correct the number.	AS EXPECTED The one refusal on this tab that is actually worded — compare IS-07.	PASS 42.0 s	
IS-07 line 348	A line with no service is refused, and the tab says so	Signed in; a patient picked; nothing picked in the Service lookup.	1. Leave the Service lookup untouched. 2. Press Enter on Qty with a perfectly good quantity of 1. 3. Read whether the cart grew and what the tab said.	Qty 1, no service	The line is refused *and* the nurse is told why — the way IS-06 is told about a bad quantity, and the way Consultancy Service answers the same mistake with `Please select fields.`	The line is correctly not taken, and nothing is said at all. The same keystroke that works every other time does nothing, with no explanation on screen. **BUG-035**	P2	KNOWN FAIL 45.8 s · BUG-035
IS-08 line 382	Enter on Qty commits the line, and the cart carries what was entered	Signed in; a priced service picked; cart empty.	1. Pick a priced service, type remarks and a quantity. 2. Press Enter on Qty. 3. Read the cart row and then the entry row back.	`quantities[0]`, 1–3; `mark` as remarks	• The cart row names the service and carries the catalogue rate, the quantity, and rate × quantity as the amount. • The commit clears the service, the remarks, the rate and the quantity for the next line.	AS EXPECTED The two date boxes are *not* cleared — IS-21.	PASS 36.0 s	
IS-09 line 421	The cart bin takes a line off, and Total Amount drops with it	Signed in; two lines in the cart.	1. Commit two lines of the same priced service in different quantities. 2. Assert Total Amount is the sum of the two. 3. Click the bin on the first row.	Quantities `n` and `n+1`, so the survivor can be told from the one removed; each line checked against what the cart says it cost	• Exactly one row goes, and it is the one whose bin was clicked. • Total Amount drops to the remaining line's amount.	AS EXPECTED	PASS 1.1 m	

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-10 line 470	Total Amount is the sum of the cart	Signed in; a priced service available.	1. Commit one line per quantity in the run's data. 2. Assert Total Amount after *each* line, not only at the end. 3. Add the cart's own amounts up and compare.	`quantities`, two lines of 1–3	Total Amount equals the running sum at every step, and the cart's rows add up to it.	AS EXPECTED Checked per line so a total that drifts is caught on the line that broke it.	PASS 1.1 m	
IS-11 line 508	The Remarks typed on a line reach the cart row	Signed in; a free service picked.	1. Type a unique mark into Remarks. 2. Commit the line. 3. Read the cart row's Remarks cell.	`mark` — unique per run	The cart row carries exactly what was typed.	AS EXPECTED Remarks is the only free-text field on the tab that reaches the grid, so it is the only thing a nurse — or a case — can mark a line with.	PASS 34.7 s	
IS-12 line 531	The service window typed on a line reaches the cart row	Signed in; a free service picked.	1. Set Start Date Time and End Date Time to a sane window earlier today. 2. Assert both reached the hidden inputs the form posts. 3. Commit the line and read the row's Start Date and End Date cells.	`window` = today 09:00 → 11:00	Both ends reach the cart row and name the same moments that were typed.	AS EXPECTED Compared as moments, not strings: the grid reprints both as `M/d/yyyy h:mm:ss AM/PM`.	PASS 36.2 s	
IS-13 line 568	The cart is emptied when another patient is picked	Signed in; one line entered for the patient in bed A.	1. Pick the patient in bed A and commit one line. 2. Without pressing Save, click the patient in bed B. 3. Wait for the pane to settle, then read the cart and Total Amount.	Two beds read live off the ward list	Picking a patient starts that patient's bill — the cart is emptied and Total Amount returns to zero, because nothing in it belongs to the patient now on screen.	The panel moves to B and the cart does not. A's services are still listed, and A's money is still in Total Amount, under B's name. **Press Save from here and B is billed for A's services. BUG-024**	P1	KNOWN FAIL 47.2 s · BUG-024
IS-14 line 631	The cart does not outlive leaving the tab	Signed in; one line in the cart.	1. Pick a patient and commit one line. 2. Click away to Medicine Indent, then back to IPD Service Entry. 3. Read the settled cart and the patient panel.	`mark` as remarks, quantity 1–3	A cart with no patient against it should not exist — either both survive the round trip or neither does.	The round trip *does* reset the patient panel — Inv.No goes back to 0, which is the half the tab gets right — and leaves the cart holding its lines, with a figure in Total Amount, against nobody at all. The next nurse to pick a patient inherits them, which is BUG-024 by another route. **BUG-025**	P1	KNOWN FAIL 41.2 s · BUG-025

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-15 line 678	The catalogue rate cannot be typed over on the form	Signed in; a priced service picked.	1. Pick a service the catalogue prices above zero. 2. Type 1 into the Rate box. 3. Commit a line of quantity 2 and read the row back.	Rate typed *down* to 1, never up — see §5 Data safety	The rate belongs to the catalogue. The box is read-only at the nurse station, and the line carries the catalogue rate whatever was typed.	The box is editable and the line carries the typed rate: a service the catalogue prices at 1000 goes into the cart at 1, and the amount follows. **BUG-026**	P1	KNOWN FAIL 48.4 s · BUG-026
IS-16 line 717	Service Change comes off the amount	Signed in; a service priced above the discount.	1. Pick a service costing more than the discount. 2. Type the discount into the Service Change box. 3. Commit the line; read the row, the box and Total Amount.	`discount` = 25	The discount reaches the line's Service Change cell and comes off the amount, and the box keeps what was typed.	Nothing reads it. The cart's Service Change cell stays 0, the amount is undiscounted, and the commit *also wipes the box* — so a nurse looking back at it sees a zero and may well type it again. **BUG-030**	P2	KNOWN FAIL 41.0 s · BUG-030
IS-17 line 764	The Service Date chosen survives committing the line	Signed in; a free service picked.	1. Read what the Service Date box opened on. 2. Set it three days back and assert it holds through the whole of the entry. 3. Press Enter on Qty, then read the box once it has stopped moving.	`backdated` = three days ago, 10:00	A service performed three days ago can be entered as three days ago — the date survives the commit.	The box takes the back-date and keeps it right up to the Enter, then silently resets to the moment of the commit. A late entry is recorded as having happened now. **BUG-029**	P2	KNOWN FAIL 21.4 s · BUG-029
IS-18 line 829	A committed line carries the schedule its column is for	Signed in; a free service picked.	1. Count the fields labelled "Schedule" on the entry form. 2. Commit a line. 3. Assert the cart's Schedule column exists, and read the row's Schedule cell.	None — the case is about what the form does not offer	A column on the cart grid can be filled by something on the form.	There is no Schedule control on this tab at all — not a lookup, not a select, not a box — so the column is empty on every line ever entered. Consultancy Service has the control and offers On Time / Off Time; this tab inherited the column without it. **BUG-034**	P3	KNOWN FAIL 39.7 s · BUG-034
IS-19 line 876	Picking a service fills the Code box	Signed in; a patient picked.	1. Assert the Code box is empty on arrival. 2. Pick a service out of the lookup and read the Code box. 3. Type a made-up code into it and commit a line anyway.	`code` = "IS-CODE-*n*"	Code sits beside the Service lookup and holds the catalogue code of whatever was picked — the one field that could tie a line to a catalogue entry.	Nothing fills it. It stays empty, stays editable, and a made-up code commits a line without a word — free text nobody sets and nothing reads. **BUG-033**	P3	KNOWN FAIL 36.7 s · BUG-033

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-20 line 914	An End Date Time before the Start Date Time is refused	Signed in; a free service picked.	1. Set Start Date Time to 18:00 and End Date Time to 08:00 the same day. 2. Press Enter on Qty. 3. Read whether the cart took it and what was said.	`reversed` = today 18:00 → today 08:00	A window whose end precedes its start is not a service window; the line is refused and the nurse is told.	Taken without a word. The row goes into the cart reading 6:00 PM → 8:00 AM, and any duration read off it is negative. **BUG-031**	P2	KNOWN FAIL 38.3 s · BUG-031
IS-21 line 956	The service window is cleared with the rest of the line	Signed in; one line committed with a window.	1. Set both date boxes and commit a line. 2. Assert the service was cleared, so the case is reading a committed row. 3. Read both date boxes.	`window` = today 09:00 → 11:00	The commit clears the whole entry row, the two date boxes included — as it already does for the service, the remarks, the rate and the quantity.	Both boxes keep their values, so the next service entered for this patient silently inherits the window of the last one unless the nurse notices and clears them by hand. **BUG-032**	P3	KNOWN FAIL 41.3 s · BUG-032
IS-22 line 997	A lookup search that matches nothing says so	Signed in; a patient picked.	1. Search the Service lookup for a term no service can match. 2. Assert no selectable row comes back. 3. Read the empty-state row's text.	`noSuchTerm` — unique per run	The lookup says there is no such service.	Nothing selectable comes back, which is right, but what the nurse is shown is an empty bordered row with no text in it — the same picture as a lookup that has not finished loading. **BUG-036**	P3	KNOWN FAIL 34.8 s · BUG-036
IS-23 line 1025	The bill lists every service charged, however many there are	Signed in. Skips if no patient in the first eighteen beds has fifteen services on the bill.	1. Walk the ward, reading each patient's bill-row count. 2. Stop at the first patient whose bill is at the cap. 3. Assert there is no pager, page-size box or total anywhere on the tab.	Up to 18 beds read live; `BILL_ROW_CAP` = 15	The bill lists every service charged, or says that it does not.	The list stops at fifteen and says nothing about it. A sixteenth service is charged and then cannot be seen on this tab at all — and since the row carries no control either (IS-24), it cannot be reached from here by any route. **BUG-028**	P2	KNOWN FAIL 16.4 s · BUG-028
IS-24 line 1077	A service saved by mistake can be taken off the bill	Signed in. Skips if no patient in reach has a service on the bill.	1. Find a patient who has been charged something. 2. Count the controls on the first bill row — buttons, links, checkboxes. 3. Compare against the cart grid, whose rows do carry a bin.	Any bed with a non-empty bill	A service charged to the wrong patient, at the wrong rate or in the wrong quantity can be cancelled, corrected or removed — as Consultancy Service allows, through a cancel-with-reason dialog recorded in IPD Bill Change History.	A bill row carries nine cells and no control of any kind. The same admission's services entered here are permanent as far as this screen is concerned. This is also why no case in this suite saves. **BUG-027**	P1	KNOWN FAIL 27.0 s · BUG-027

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-25 line 1123	Save refuses an empty cart	Signed in; a patient picked; cart asserted empty.	1. Count the bill rows. 2. Assert the cart is empty — if it were not, this click would charge a real patient. 3. Click Save and read the message; re-count the bill.	None	Save refuses with `Please add at least one service.` — the wording until 22 Oct 2026 was `Please input items` — and writes nothing.	AS EXPECTED The only Save this suite presses, and the emptiness of the cart is asserted rather than assumed.	PASS 11.8 s	
IS-26 line 1151	The bill grid names its columns	Signed in; a patient picked.	1. Read the bill grid's column headers. 2. Assert the end-of-window column is named.	None	The eighth column reads `End Date` — the name the cart grid uses for the same value one grid up.	AS EXPECTED since 22 Oct 2026 — **BUG-037 fixed**, with `S.Date` and `Served_By` tidied in the same pass. The case had a fault of its own as well: it read the headers with `allInnerTexts()`, which applies the CSS that uppercases them, so it was comparing `END DATE` against `End Date` and could never have passed. It now reads `allTextContents()`, which is what IS-01 was doing all along.	P3	PASS 8.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	PRI	STATUS
IS-27 `line 1184`	Saved services carry the window they were entered with	Signed in. Skips if no patient in reach has a service on the bill.	1. Walk up to eight beds, reading each bill in full. 2. Count the saved rows carrying either end of a service window. 3. Record the figure as an annotation.	Up to 8 beds read live	A figure, not a verdict — see the Actual column.	OBSERVATIONAL — the 06 Oct run read **0 of 46** saved services carrying either end of a window, across four beds (S.Care-06 0/15, Free Bed Male-515 0/5, Free Bed-609 0/11, S.Care-63 0/15). Deliberately not marked `test.fail()`. Both window boxes are optional on the entry form, so a bill of services entered without one is telling the truth, and proving a *saved* window is dropped would mean saving — which this tab offers no way to undo (IS-24). Nought out of forty-six, against a cart grid that shows both ends (IS-12), is a figure for the module owners to explain, not a defect this suite is in a position to raise. Listed under Known coverage gaps.	P3	PASS 1.0 m

21. Known coverage gaps

GAP	WHY IT IS OPEN	SUGGESTED NEXT STEP
Grid search and row actions on the patient list (the old TC_REG_005–007)	Removed from the spec; the cases keyed off live data that was not stable enough to assert on.	Re-add against a patient the test registers itself, the way TC_FLOW_001 does.
Advance payment reaching the admission	Observed failing; see BUG-002. Asserting it today would make TC_FLOW_001 fail for a defect that is already tracked.	Add the assertion as soon as BUG-002 is fixed, so it guards the fix.
What is inside a printed admission document	TC_ADMD_007 checks that each Action button produces a real, non-empty PDF, not what the PDF says. Reading the text back would mean parsing it, and nothing in the suite does that today.	Add a PDF text extractor and assert the patient's name and admission number appear in each document.
The dashboard's list-level Print List / Print Package List, and the per-row cloud-arrow-down button	The two list buttons print the whole filtered page and are not tied to one admission. The cloud button produced nothing observable, and what it is meant to do is unknown.	Ask the module owners what the cloud button does, then cover all three.
Create-patient field validation (required fields, mobile/NID format, duplicate NID)	Every negative case would need the Confirm button, which writes to live data.	Needs a test tenant or a reset endpoint before it can be automated safely.
Firefox, WebKit and mobile viewports	Projects are present in the config but commented out — the host drops sessions under concurrent browsers.	Enable once the host tolerates more than one worker, or run them in separate scheduled passes.
Roles other than Administrator	Only one account is configured. Doctor, receptionist and accountant are declared as types but never exercised.	Add per-role accounts to `.env` and a storage state per role.
What the pharmacy and the canteen do next with an indent	The suite files indents and confirms they arrive as Pending. Partially Served, Served and Cancelled are read as tab labels (TC_NS_011) and never driven — moving an indent through them on a live ward would commit stock the hospital holds.	Cover the Verify Indent actions once there is a tenant that is not the hospital's own.
Consultancy services billed past the fifteen-row cap	CS-23 establishes the cap exists; nothing tests what a sixteenth charge does, because it cannot be read back or cancelled from the tab afterwards (BUG-016).	Cover it from the Accounts side, where the bill is presumably complete.
The IPD Service Entry *Save* itself — and everything downstream of it	No case on that tab saves, on purpose. Its bill grid carries no bin, no edit and no cancel (IS-24 / BUG-027), so a service saved there cannot be taken off again from the tab — a case that pressed Save would make a permanent change to a live patient's bill with no way to undo it. IS-25 presses Save only against a cart it has asserted is empty.	Cover the save once BUG-027 gives the tab a cancel path, or from a tenant that is not the hospital's own. Until then the save is exercised by hand, not by the suite.

GAP	WHY IT IS OPEN	SUGGESTED NEXT STEP
Whether a saved IPD service keeps its Start / End window	IS-27 reads every bill in reach and has yet to find one saved service carrying either end of a window, against a cart grid that shows both (IS-12). But the two boxes are optional, so a bill of services entered without one is telling the truth, and proving the save drops them would mean saving — see the row above. The case records the figure and asserts nothing.	Ask the module owners whether the window is meant to persist. If it is, this becomes a defect; if it is not, the two columns on the bill grid should go.
IPD services charged past the fifteen-row cap	IS-23 establishes the cap exists on this tab too; nothing tests what a sixteenth charge does, because it can be read back from neither grid nor cancelled from the tab (BUG-028 with BUG-027).	Cover it from the Accounts side, where the bill is presumably complete.
The IPD Service Entry *OTName* picker and *Print All*	OTName lists several hundred OT procedures and nothing in the suite establishes what picking one changes about a saved entry — which cannot be checked without saving. Print All prints the whole cart and is not tied to one line.	Cover both with the save, once there is a cancel path or a non-production tenant.
Diet Dashboard rows past 120, and the dashboard's own date boxes	DD-07 shows the grid cuts at 120 with no pager. The date boxes are read through the hidden inputs the app posts, because the visible flatpickr never mirrors them (BUG-019) — so no case asserts on what a nurse can actually see there.	Re-point the date assertions at the visible boxes once BUG-019 is fixed.
OPD, Pharmacy, Diagnostic, HR, Accounts	Out of scope for this cycle — only their tiles are checked on the launcher.	Plan per-module suites in the next cycle.